uc3m | Universidad Carlos III de Madrid

Doble Grados de Ingeniería Informática y ADE
2025-2026

Trabajo Fin de Grado

“Automatización del análisis táctico en fútbol sala a partir de videos”

Jaime Ballesteros Calvo

Tutor: Eduardo Cibrian Sánchez

Resumen

El punto de partida de este proyecto es una problemática estructural en el fútbol sala no profesional: la desconexión tecnológica. Mientras la élite basa su rendimiento en métricas precisas, la inmensa mayoría de clubes amateurs carece de herramientas de análisis debido a barreras económicas y de infraestructura, dependiendo casi exclusivamente de grabaciones manuales con teléfonos móviles que resultan ineficientes para un análisis táctico objetivo. Para resolver esta carencia, se planteó el desarrollo de FutsalVision con una premisa de ingeniería clara: sustituir el hardware costoso y las cámaras fijas por un procesamiento de software capaz de interpretar vídeos monoculares estándar.

El desarrollo técnico comenzó abordando el desafío de la visión artificial. Las pruebas iniciales demostraron que la detección tradicional por recuadros (*bounding boxes*) era insuficiente para determinar la posición real de un jugador en la pista. Por ello, se evolucionó hacia el uso de YOLOv8 con Segmentación de Instancias, lo que permitió identificar la silueta exacta de los jugadores.

Una vez resuelta la detección, el proyecto se enfrentó al problema matemático de la proyección en vídeos con movimiento y zoom variable, donde las homografías clásicas fallaban. La solución fue el diseño de un Proyector Geométrico Híbrido. Este algoritmo propio utiliza referencias visuales estáticas del campo, como las líneas de las áreas y el medio campo, para "anclar" la imagen y corregir la perspectiva dinámicamente, permitiendo traducir los píxeles del vídeo a coordenadas métricas reales en un tablero 2D.

Finalmente, para completar la herramienta, se implementó una capa de lógica táctica. El sistema no se limita a proyectar lo que ve la cámara, sino que reconstruye la coherencia del juego: identifica la alineación inicial en un "Frame 0" y utiliza la proyección temporal para inferir la posición de los jugadores que quedan momentáneamente fuera de plano. El resultado es un prototipo funcional validado con una precisión de detección superior al 91%, demostrando que es viable generar telemetría fiable en entornos no controlados.

Palabras clave: Visión por Computador, Fútbol Sala, YOLOv8, Telemetría Deportiva, Proyección Geométrica

Índice general

1. Introducción 8
2. Estado del Arte 10
2.1 Ámbito General 10
2.2 Análisis del mercado actual 13
2.3 Tecnologías utilizadas 17
3. Análisis y Diseño del Sistema 23
3.1 Definición del problema y dificultades 23
3.2 Requisitos del sistema 25
3.2.1. Requisitos Funcionales (RF) 25
3.2.2. Requisitos No Funcionales (RNF) 27
3.3 Cómo se interactúa con el sistema 28
3.3.1. Experiencia de Usuario: El concepto "Caja Negra" 28
3.3.2. Formalización Técnica: Diagrama de Casos de Uso (UML) 30
3.4 Arquitectura de Componentes 32
4. Implementación Técnica 34
4.1. Gestor de Datos 35
4.1.1. Adquisición y Unificación de Datasets Heterogéneos 35
4.1.2. Definición de Clases y Refinamiento del Etiquetado 36
4.1.3. Aprendizaje Asistido y Adaptación de Dominio 37
4.1.4. Preprocesamiento y Aumento de Datos (Data Augmentation) 39
4.1.5. Configuración del Entrenamiento y División del Dataset 40
4.1.6. Definición del Caso de Uso Real (MVP) 41
4.2. Motor de Inferencia (La Visión) 42
4.2.1. Ejecución sobre el Caso de Uso (MVP) 43
4.2.2. Serialización de Datos y Metadatos (.txt) 43
4.2.3. Visualización y Verificación de Resultados 44

4.3. Analizador Semántico y Reconstrucción Geométrica 44
4.3.1. División Dinámica y Asignación de Lados 45
4.3.2. Persistencia Temporal y Generación de Datos Semánticos 46
4.4. Proyector Geométrico (Las Matemáticas) 46
4.4.1. Calibración Estática: Localización de la Cámara 47
4.4.2. Proyección Dinámica: El Cono de Visión (FOV) 50
4.4.3. Mapeo de Entidades: Algoritmos de Proyección y Desafíos Técnicos 56
4.5. Sintetizador Táctico: Consolidación y Generación de Entregables 60
4.5.1. La Estrategia del "Frame 0" y la Mirada al Futuro 61
4.5.2. Implementación Técnica: El Código AnalizadorTactico 62
5. Pruebas y Resultados 65
5.1. Primera Prueba: Validación Piloto 66
5.2 Segundo y tercer entrenamiento en Roboflow 68
5.3. Entrenamiento completo en Google Colab 71
5.4. Transformación al Tablero y Validación Geométrica 73
5.4.1 El "Golden Standard": Frame 0 74
5.4.2 Análisis Dinámico Temporal 74
6. Planificación 78
7. Presupuesto 81
7.1 Costes de Personal 82
7.2. Amortización de Hardware 83
7.3. Costes de Software y Servicios Cloud 83
7.4. Resumen y Presupuesto Final 84
8. Marco Legal y Propiedad Intelectual 85
8.1. Protección de Datos y Privacidad (RGPD) 86
8.2. Licenciamiento de Software y Limitaciones Comerciales 86
9.1. Democratización del Dato Deportivo (Mercado) 87
9.2. Alineación con la Agenda 2030 (ODS) 88
10. Conclusiones y Pasos Futuros 89
10.1 Líneas futuras 90

11. Anexo 92
Anexo A: Imágenes Complementarias del Proceso. 92
Anexo B: Repositorios de Código y Datasets 93
B.1 Github 93
B2. Roboflow 93
12. Bibliografía..... 94
13. Declaración de IA..... 95

Índice de figuras y tablas

Ilustración 1. Diagrama del Usuario 29
Ilustración 2. Diagrama idílico del sistema 29
Ilustración 3. Diagrama real del sistema 30
Ilustración 4. Diagrama de Casos 31
Ilustración 5. Diagrama UML 33
Ilustración 6. Opción A, Bounding Boxes 36
Ilustración 7. Opción B, Instance Segmentation 36
Ilustración 8. Flip Horizontal 40
Ilustración 9. Blur 40
Ilustración 10.Brightness / Exposure 40
Ilustración 11. Rotation 40
Ilustración 12. Cono de Telemetría 55
Ilustración 13. Vídeo de Salida Con Debugs 60
Ilustración 14. Vídeo de Salida Sin Debugs 61
Ilustración 15. Frame 0 64
Ilustración 16. Visualización del lote de validación 67
Ilustración 17. Vídeo procesado con el primer Modelo 68
Ilustración 18. Análisis detallado por clase 69
Ilustración 19.Visualización del lote de test 70
Ilustración 20. Vídeo procesado con el Segundo Modelo 72
Ilustración 21. Vídeo procesado con el Tercer Modelo 73
Ilustración 22. Imagen al segundo 5 74
Ilustración 23. Imagen al segundo 10 75
Ilustración 24. Imagen al segundo 15 76
Ilustración 25. Imagen al segundo 20 76
Ilustración 26. Imagen al segundo 25 77
Ilustración 27. Imagen al segundo 29 77
Ilustración 28. Gantt Chart 81
Ilustración 29. Gantt Chart Primeras Semanas 92
Ilustración 30. Gantt Chart Segundas Semanas 92

Ilustración 31. Gantt Chart Terceras Semanas 92
Tabla 1. Coste Personal 83
Tabla 2. Amortización de Hardware..... 83
Tabla 3. Costes de Software y Servicios Cloud 84
Tabla 4. Presupuesto Final..... 85

1. Introducción

La historia de la evolución siempre ha ido de la mano del uso de herramientas para superar las diversas dificultades que se van encontrando. Hoy en día, la herramienta principal es, sin ninguna duda, la inteligencia artificial. Estamos ante una tecnología que no solo está redefiniendo sectores económicos enteros, sino que se ha convertido en el motor del desarrollo social futuro. Su alcance es total, abarcando desde asistentes generalistas que ya forman parte de nuestra rutina hasta soluciones de nicho altamente específicas capaces de procesar información compleja en segundos. Si nos fijamos en el deporte, este impacto es innegable. En la élite profesional, el dato manda; la gestión basada en estadísticas como se puede ver en películas basadas en ejemplos reales como *Moneyball,* demuestran que el análisis de datos es ya indispensable para optimizar el rendimiento y tomar decisiones estratégicas.

No obstante, esta ola de digitalización choca con un muro cuando miramos hacia disciplinas como el fútbol sala, especialmente fuera del circuito profesional. Existen barreras estructurales que frenan la entrada de tecnología avanzada en los clubes modestos: la mayoría juega en pistas municipales donde es imposible instalar sistemas de cámaras fijas, la normativa prohíbe el uso de dispositivos electrónicos corporales (*wearables)* por seguridad y, sobre todo, la realidad económica hace inviable el pago de licencias de software de élite. El resultado es que los equipos que quieren analizarse acaban dependiendo de métodos rudimentarios, grabando con el móvil y revisando el vídeo de forma manual y subjetiva, un proceso lento que suele depender del escaso tiempo libre del entrenador.

Esta carencia no es una simple suposición, sino una realidad cuantificada. En la investigación de mercado realizada paralelamente en el Trabajo de Fin de Grado de Administración y Dirección de Empresas (Ballesteros Calvo, 2026) vinculado a este proyecto, se detectó una brecha tecnológica crítica: el 79,5% de los equipos amateur no dispone de ninguna herramienta de registro audiovisual. Este estudio reveló una

correlación directa entre la falta de tecnología y la frustración de los jugadores, quienes perciben que su progresión táctica se estanca por la falta de *feedback* objetivo. Existe, por tanto, una necesidad latente y validada de una solución que profesionalice el análisis sin requerir grandes infraestructuras.

De esta necesidad nace la propuesta de este trabajo: el desarrollo de **FutsalVision**, un sistema basado en visión por computador y *Deep Learning* diseñado para romper estas barreras. El objetivo es crear una herramienta capaz de procesar un vídeo estándar de un partido —grabado con un simple teléfono móvil— y realizar un reconocimiento automático de todos los elementos del juego. Mediante algoritmos de transformación de perspectiva, el sistema traslada la posición de jugadores y balón a un tablero táctico 2D virtual, generando una vista cenital (simulando una cámara situada justo encima del campo) que elimina las distorsiones y el caos visual de la grabación original. De esta forma, se busca democratizar el acceso al análisis táctico, ofreciendo a entrenadores y jugadores una visión clara y estructurada del juego que, hasta ahora, estaba reservada exclusivamente a los grandes clubes.

Bajo este marco, el objetivo principal que persigue el trabajo es el diseño e implementación integral de un sistema de visión por computador capaz de interpretar la dinámica del juego a partir de grabaciones monoculares no controladas. La finalidad última es la **generación automática de una representación táctica cenital, traduciendo la imagen bruta del vídeo en información posicional estructurada sobre un tablero virtual**, permitiendo así una lectura del juego libre de distorsiones visuales.

Para la consecución del objetivo general, el trabajo se desglosa en los siguientes hitos técnicos y funcionales:

- **Implementar y ajustar un modelo de detección**: Configurar y entrenar una red neuronal basada en la arquitectura YOLOv8 (Jocher et al., 2023) para la identificación precisa de elementos dinámicos (jugadores, balón) y estáticos (puntos clave de la pista) en entornos no controlados.

- **Desarrollar algoritmos de transformación de perspectiva**: Diseñar el módulo matemático necesario para realizar la transformación entre el vídeo grabado por la cámara y el plano cenital del modelo táctico.
- **Proyectar espacialmente los datos**: Implementar la lógica de mapeo que traduzca las coordenadas de detección del vídeo original a un sistema de coordenadas métricas normalizadas sobre un tablero virtual.
- **Generar la visualización táctica**: Construir una interfaz de salida gráfica que represente la posición y movimiento de los elementos en un esquema de campo 2D, eliminando la distorsión visual y facilitando el análisis táctico

Finalmente, para facilitar la comprensión global del proyecto, la presente memoria se estructura siguiendo el ciclo de vida del desarrollo de software. Tras esta **Introducción**, se profundiza en el **Estado del Arte** y el contexto tecnológico, sentando las bases teóricas que justifican las decisiones de **Análisis y Diseño del Sistema**. El núcleo del documento se centra en la **Implementación Técnica**, detallando los algoritmos de detección y transformación desarrollados, para posteriormente validar su funcionamiento en el apartado de **Pruebas y Resultados**. El trabajo se completa con los aspectos de gestión —abarcando la Planificación, Presupuesto y Marco Legal— y culmina con las Conclusiones, donde se sintetizan los logros alcanzados y se proponen líneas de evolución futura.

2. Estado del Arte

2.1 Ámbito General

La reducción de la incertidumbre en los procesos de toma de decisiones ha constituido históricamente uno de los desafíos fundamentales para el avance de la ingeniería de sistemas y la ciencia aplicada. Si bien en etapas previas la humanidad se apoyó en herramientas cognitivas externas básicas —como la cartografía analógica o los registros escritos— para interpretar su entorno, la revolución digital y el abaratamiento de los semiconductores han transformado esta necesidad en sistemas complejos de

procesamiento de información capaces de objetivar la realidad física. Esta evolución tecnológica es especialmente palpable en sectores punteros como **el marketing y el retail**, industrias que han actuado como punta de lanza en la adopción de sensores para la monitorización del comportamiento humano. Analizar cómo estos sectores han resuelto el problema de transformar imágenes no estructuradas y señales de radiofrecuencia en métricas de negocio precisas resulta indispensable para comprender los cimientos arquitectónicos sobre los que se construye la propuesta de este trabajo.

En el ámbito específico del análisis de espacios físicos comerciales, compañías referentes como **RetailNext** (RetailNext, s.f.) han establecido un nuevo paradigma técnico al superar las limitaciones inherentes a los métodos declarativos tradicionales —como las encuestas o el conteo manual— mediante el despliegue de arquitecturas híbridas de Internet de las Cosas (IoT) y visión por computador estereoscópica. La arquitectura de hardware de soluciones como las de RetailNext no se limita a la instalación de cámaras de videovigilancia estándar, sino que integra sensores ópticos avanzados capaces de percibir la profundidad mediante disparidad binocular, emulando la visión humana. Estos sensores generan mapas de profundidad que permiten al sistema distinguir volumétricamente entre un ser humano, un carrito de la compra o un elemento estático del mobiliario, filtrando así falsos positivos que contaminarían la muestra de datos y garantizando la integridad de la información recolectada.

Paralelamente al análisis visual, estos sistemas implementan una capa de detección por radiofrecuencia que actúa como un mecanismo de validación cruzada. Mediante la instalación de nodos receptores distribuidos estratégicamente en el local, la infraestructura realiza una triangulación continua de los dispositivos móviles de los clientes. El principio técnico subyacente se basa en la medición de la intensidad de la señal recibida (RSSI) de las tramas de los protocolos Wi-Fi y Bluetooth Low Energy (BLE). Al analizar la atenuación de la señal desde tres o más puntos de acceso conocidos, el sistema resuelve ecuaciones de trilateración para estimar las coordenadas (x, y) del sujeto en el plano de la tienda con un margen de error reducido. A nivel de procesamiento de datos, esta información bruta se somete a algoritmos de filtrado que discriminan entre el personal de la tienda y los clientes reales. La fusión de estos dos flujos de datos —la trayectoria visual y la huella digital de radiofrecuencia— permite calcular métricas de comportamiento de alto nivel, como el *dwell time* o tiempo de permanencia. Este

indicador no es un simple cronómetro; se deriva de algoritmos de lógica difusa que determinan si una parada en una coordenada específica corresponde a una interacción de interés con un producto o a una simple espera, generando mapas de calor (*heatmaps*) que representan la densidad de ocupación espacial mediante la acumulación de vectores de posición a lo largo del tiempo.

En una línea complementaria, profundizando aún más en la micro-monitorización del comportamiento humano, empresas como **Tobii** Pro (Tobii, s.f.) han liderado la implementación de tecnologías de *Eye-Tracking* o seguimiento ocular, llevando la objetivación de la atención a un nivel biométrico. El fundamento técnico de estas soluciones se aleja de la óptica convencional para adentrarse en la iluminación infrarroja. El hardware de Tobii utiliza proyectores de luz infrarroja cercana (NIR) que emiten patrones invisibles al ojo humano hacia el rostro del usuario. Estos rayos generan dos tipos de reflejos críticos para el sistema: el reflejo corneal (o primera imagen de Purkinje) y la pupila oscura o brillante, dependiendo de la configuración del iluminador. Cámaras de alta velocidad capturan estos reflejos a frecuencias que pueden superar los 100 Hz, enviando las imágenes a una unidad de procesamiento que ejecuta algoritmos de geometría 3D para calcular la relación vectorial entre la posición de la córnea y el centro de la pupila.

Dado que el ojo humano no es una esfera perfecta, el sistema debe realizar una calibración inicial para modelar la fisiología específica del usuario. Una vez calibrado, el algoritmo proyecta un vector tridimensional desde el ojo hacia el plano del estímulo (ya sea una pantalla o un estante físico). El resultado es un mapeo píxel a píxel que determina exactamente qué punto del entorno está siendo procesado por la fóvea, la parte de la retina responsable de la visión detallada. Esta tecnología permite generar métricas como el tiempo hasta la primera fijación o la duración de la visita visual, demostrando que la ingeniería actual dispone de la madurez necesaria para convertir procesos subconscientes, rápidos y aparentemente caóticos en conjuntos de datos estructurados y auditables. Es precisamente este principio de conversión de lo analógico y subjetivo a lo digital y objetivo lo que FutsalVision pretende trasladar desde el entorno controlado del laboratorio de marketing al escenario dinámico, impredecible y técnicamente hostil de una pista de fútbol sala.

2.2 Análisis del mercado actual

Una vez establecido el marco tecnológico general sobre cómo la ingeniería moderna aborda el seguimiento humano, es necesario descender al dominio específico del fútbol sala para evaluar las herramientas existentes. El análisis del fútbol sala se estructura en una arquitectura de datos de tres capas: la adquisición de la señal física mediante sensores, la normalización y transmisión de la información, y el procesamiento algorítmico para la interpretación táctica. En la élite profesional, representada por ecosistemas de innovación como el **Barça Innovation Hub (Barça Innovation Hub, s.f.)**, esta arquitectura se encuentra altamente optimizada, combinando inputs subjetivos recogidos a través de aplicaciones móviles (cuestionarios de bienestar o RPE) con telemetría objetiva de alta precisión. Sin embargo, la transferencia de este modelo tecnológico al fútbol sala no profesional se enfrenta a barreras de infraestructura y costes que las soluciones actuales no han logrado democratizar eficazmente.

En el segmento de la monitorización física individual, el estándar industrial lo constituyen los Sistemas Electrónicos de Seguimiento y Rendimiento (EPTS), comercializados por firmas como **OLIVER Sports (OLIVER Sports, s.f.)**. Desde una perspectiva de ingeniería electrónica, el corazón de estos dispositivos —que suelen insertarse en medias o petos compresivos— es una Unidad de Medición Inercial (IMU) de 9 ejes. Este componente microelectromecánico (MEMS) fusiona tres sensores distintos en un mismo chip de silicio: un acelerómetro triaxial, que mide las fuerzas inerciales y gravitatorias para cuantificar impactos, arranques y frenadas; un giroscopio, que mide la velocidad angular para detectar rotaciones y cambios de dirección; y un magnetómetro, que actúa como una brújula digital referenciando el norte magnético para determinar la orientación del jugador en el espacio.

El desafío técnico de estos dispositivos no reside tanto en la captura del dato, sino en el procesamiento de la señal. Los sensores MEMS son intrínsecamente ruidosos y propensos a la deriva (*drift*) con el tiempo. Para solucionar esto, el firmware de los dispositivos ejecuta algoritmos de fusión de sensores, típicamente Filtros de Kalman Extendidos (EKF) o filtros de Madgwick, que predicen y corrigen el estado del sistema en tiempo real, combinando la estabilidad a corto plazo del giroscopio con la referencia absoluta del

acelerómetro y el magnetómetro. Gracias a este procesamiento matemático, los dispositivos pueden entregar métricas de carga interna y externa, como el *PlayerLoad* o la potencia metabólica. Sin embargo, existe una limitación física insalvable para estos dispositivos en el contexto del fútbol sala: la geolocalización en interiores. En deportes al aire libre, la posición (x, y) se obtiene mediante receptores GNSS que triangulan señales de constelaciones satelitales (GPS, GLONASS, Galileo). En un pabellón cubierto, la estructura metálica y de hormigón actúa como una jaula de Faraday parcial. Para obtener posicionamiento en estas condiciones, la ingeniería actual obliga al despliegue de Sistemas de Posicionamiento Local (LPS). La tecnología más eficiente para ello es la Banda Ultra Ancha (UWB), que calcula la posición midiendo el Tiempo de Vuelo (ToF) o la Diferencia de Tiempo de Llegada (TDoA) de pulsos de radio entre el dispositivo del jugador y una serie de antenas o "anclas" fijas instaladas en el perímetro de la pista. Aunque técnicamente superior al GPS en precisión, la necesidad de instalar, calibrar y mantener una infraestructura física de antenas supone una barrera de entrada económica y logística absoluta para clubes amateurs, dejando a los equipos modestos "ciegos" tácticamente.

Ante esta carencia de datos posicionales automáticos, el mercado se ha apoyado tradicionalmente en la digitalización manual. Grandes proveedores de datos como **Opta** han industrializado este proceso mediante "granjas de datos" donde operadores humanos visualizan partidos y registran eventos. Tecnológicamente, estos sistemas no poseen inteligencia; son interfaces de entrada de datos que alimentan bases de datos relacionales (SQL) o documentales (NoSQL). La estructura de la información se basa en esquemas XML o JSON donde cada evento es una entidad con atributos: ID de jugador, coordenadas (x, y) aproximadas por clic manual, *timestamp* y tipo de acción. Herramientas de software de escritorio como **Nacsport, LongoMatch** o **Eric Sports** siguen esta misma filosofía. Su arquitectura de software funciona como un indexador temporal de archivos multimedia. No "leen" los píxeles del vídeo; simplemente crean un archivo lateral con metadatos que apuntan a momentos específicos de la línea de tiempo. El cuello de botella de esta tecnología es el factor humano: el análisis es lineal y no escalable. Procesar un partido requiere un tiempo igual o superior a la duración de este, ya que esos eventos y la creación de esos metadatos se tienen que hacer de manera manual, lo que hace inviable su uso masivo en categorías base donde los entrenadores son voluntarios con tiempo limitado.

Como respuesta a la ineficiencia manual y a la complejidad de los sensores físicos, ha emergido una nueva generación de sistemas de captura automatizada basados en visión por computador, siendo **Veo Technologies** (Veo Technologies, s.f.) el máximo exponente en este nicho. Analizar la ingeniería detrás de una cámara Veo revela una arquitectura diseñada específicamente para resolver el problema de la grabación sin operador. A diferencia de una cámara convencional con un solo sensor y lente, el hardware de Veo integra un *array* óptico dual con sensores 4K y lentes de ojo de pez (*fisheye*). Esta configuración permite capturar un campo de visión de 180 grados, abarcando la totalidad de la pista desde una posición central elevada. El verdadero núcleo tecnológico de estas cámaras reside en su pipeline de procesamiento de imagen. Dado que las lentes de gran angular introducen una distorsión de barril severa, el primer paso es la corrección geométrica. Posteriormente, el sistema debe fusionar los flujos de vídeo de las dos lentes. Esto se logra mediante algoritmos de *stitching* (cosido), que calculan matrices de homografía para alinear las imágenes en la zona de superposición, aplicando técnicas de mezcla multibanda para que la unión sea invisible al ojo humano, compensando las diferencias de exposición entre un sensor que puede estar mirando hacia una fuente de luz y el otro hacia una zona de sombra.

Sobre esta imagen panorámica unificada, el sistema ejecuta la capa de Inteligencia Artificial. Contrario a la creencia popular, la cámara no tiene partes móviles. El movimiento es una ilusión digital creada por un algoritmo de Detección de Región de Interés (ROI). Una red neuronal convolucional (CNN) entrenada con millones de fotogramas de fútbol detecta la posición del balón y la densidad de jugadores en cada instante. Con esta información, el software recorta una ventana de vídeo Full HD dentro del lienzo 4K original, desplazando suavemente este recorte para simular un paneo (*pan*) y zoom cinematográfico.

Sin embargo, esta arquitectura presenta fricciones técnicas y operativas considerables que alejan esta tecnología de la realidad del fútbol sala amateur. Al tratarse de un sistema 'caja negra' propietario, el club queda cautivo de un hardware específico —como la Veo Cam 3— que, pese a su coste elevado (ronda los 1.199 € solo el dispositivo), presenta una fragilidad estructural notable. El equipo no solo debe realizar una inversión inicial en la cámara y trípodes certificados (otros 200-400 €), sino que debe operar bajo el temor constante a perder la garantía: estos dispositivos integran sensores de fuerza G

internos que invalidan el soporte técnico si detectan impactos por caídas o balonazos directos, algo crítico considerando que la certificación IP54 del dispositivo protege contra el clima, pero no contra impactos cinéticos. A esto se suma la inviabilidad logística en pistas municipales: el fabricante exige colocar la cámara sobre trípodes elevados (de hasta 7 metros) y, crucialmente, mantener una distancia de seguridad de entre 3 y 5 metros respecto a la línea de banda para evitar puntos ciegos y daños. Esta exigencia es imposible de cumplir en la mayoría de pabellones modestos, donde la línea de banda colinda directamente con la pared o la grada o es el lugar donde se sienta el árbitro de mesa. Finalmente, la barrera económica se perpetúa mediante un modelo de negocio de suscripción recurrente obligatoria (SaaS), que añade un coste operativo fijo de unos 400 € anuales por equipo para procesar los vídeos, haciendo que el análisis táctico sea insostenible para presupuestos que a menudo dependen de subvenciones o cuotas de socios.

Finalmente, en la frontera tecnológica más próxima a FutsalVision, se encuentran las soluciones de software de *tracking* óptico puro como **Metrica Sports (Metrica Sports, s.f.)S**. Esta tecnología representa el intento de eliminar el hardware dedicado y extraer datos directamente del vídeo de retransmisión. Su base técnica se apoya en detectores de objetos de última generación y en algoritmos de geometría proyectiva. El sistema identifica a los jugadores en el plano de la imagen 2D y, mediante la detección de líneas y puntos característicos del campo, calcula una matriz de homografía que permite proyectar las coordenadas de los pies de los jugadores al plano del suelo, generando una vista cenital. A pesar de su sofisticación teórica, la aplicación práctica de estas herramientas en el fútbol sala amateur se topa con problemas de robustez en la visión artificial. Los algoritmos de Metrica, entrenados predominantemente en estadios de fútbol profesional con césped verde uniforme e iluminación perfecta, sufren una degradación de rendimiento severa en pabellones *indoor*. El suelo de parqué o goma pulida genera reflejos especulares que confunden a los detectores; la iluminación artificial a menudo produce parpadeo (*flicker*) y sombras duras; y, lo más crítico, las pistas polideportivas suelen tener pintadas líneas de baloncesto, balonmano y voleibol superpuestas, lo que hace fallar a los algoritmos de autocalibración de la cámara. Además, en planos cortos o grabaciones a pie de pista con móviles, la oclusión entre jugadores es frecuente, lo que requiere lógicas de re-identificación y *tracking* muy complejas para no perder la traza de quién es quién. Actualmente, estas herramientas siguen requiriendo intervención humana

para corregir errores de detección, lo que las aleja del objetivo de automatización total que demanda el mercado amateur. Es en esta intersección de necesidades no resueltas —automatización total, independencia de hardware y robustez en entornos *indoor* caóticos— donde se sitúa la propuesta técnica de este Trabajo de Fin de Grado.

2.3 Tecnologías utilizadas

El éxito de cualquier sistema de inteligencia artificial moderno, especialmente aquellos basados en visión por computador, no reside únicamente en la sofisticación del algoritmo matemático, sino en la calidad y variedad de los datos con los que este se entrena. Para el desarrollo de FutsalVision, esta premisa se convirtió en el primer gran desafío de ingeniería. El objetivo del proyecto es ambicioso: lograr que el sistema sea capaz de interpretar tácticamente un partido de fútbol sala independientemente del escenario. Esto implica que el modelo no puede aprender solo de imágenes perfectas de la primera división, sino que debe ser capaz de generalizar y funcionar correctamente en la heterogénea realidad del fútbol sala amateur: pistas de parqué brillante, suelos de goma azul o cemento, pabellones con iluminación deficiente o pistas exteriores con sombras duras. Para lograr esta robustez, era necesario construir una base de datos de entrenamiento (dataset) extremadamente variada y rica en casuísticas visuales.

Ante esta necesidad, la primera decisión estructural del proyecto no fue qué red neuronal utilizar, sino qué herramienta emplear para gestionar, etiquetar y procesar esa ingente cantidad de imágenes. En el mercado actual de la visión artificial, la gestión del dato se ha convertido en una industria en sí misma, ofreciendo un abanico de opciones que van desde soluciones gratuitas y locales hasta plataformas empresariales en la nube. Se realizó un análisis de mercado para evaluar qué tecnología se adaptaba mejor a los recursos de un Trabajo de Fin de Grado, buscando el equilibrio entre potencia técnica y eficiencia de tiempo.

En una primera instancia, se evaluaron las herramientas de código abierto tradicionales que se ejecutan en local, siendo los exponentes más conocidos **LabelImg** y **LabelMe**. Estas herramientas han sido durante años el estándar en la comunidad académica por ser gratuitas y no requerir conexión a internet. Sin embargo, al realizar las

primeras pruebas de concepto, se evidenciaron carencias críticas para la envergadura de este proyecto. LabelImg, por ejemplo, es excelente para dibujar cajas delimitadoras (bounding boxes) simples, pero su interfaz es rudimentaria y carece de herramientas para gestionar grandes volúmenes de datos de forma colaborativa. Buscando una mayor profesionalización, se analizó la opción de **CVAT** (Computer Vision Annotation Tool). Esta herramienta, respaldada originalmente por Intel, es extremadamente potente y es considerada un estándar en la industria para el etiquetado de vídeo. Sus capacidades técnicas son indiscutibles, pero presenta una barrera de entrada operativa muy alta: su despliegue óptimo requiere la orquestación de contenedores Docker y, preferiblemente, la configuración de un servidor dedicado. Para un proyecto individual, dedicar días a la configuración y mantenimiento de la infraestructura del servidor suponía desviar recursos valiosos del objetivo principal, que es el desarrollo del algoritmo de análisis táctico. Por último, se consideraron plataformas empresariales de élite como **Labelbox** o **SuperAnnotate**. Estas soluciones ofrecen flujos de trabajo automatizados y gestión de equipos de alto nivel, pero sus modelos de precios están orientados a grandes corporaciones, haciendo que sus licencias sean prohibitivas para el presupuesto de un estudiante universitario.

Fue en este punto intermedio, buscando democratizar las capacidades de la élite empresarial sin la complejidad de infraestructura de CVAT, donde se seleccionó **Roboflow** como la piedra angular para la gestión de datos de FutsalVision. La elección de Roboflow no fue casual, sino que respondió a su capacidad para centralizar todo el ciclo de vida del dato en una única plataforma web intuitiva. A diferencia de sus competidores, Roboflow no es solo una herramienta de dibujo, sino un ecosistema que solucionaba de raíz el problema de la "página en blanco". Uno de los mayores obstáculos al iniciar un proyecto de Deep Learning es conseguir las primeras imágenes; Roboflow cuenta con Roboflow Universe, un repositorio comunitario masivo donde otros investigadores publican sus datasets. Esto permitió no partir de cero, sino acceder a bases de datos ya existentes de fútbol y deportes de equipo, descargarlas y fusionarlas con las grabaciones propias realizadas para el TFG.

Esta capacidad de integración fue decisiva, pero trajo consigo un nuevo reto técnico: la unificación de clases. Al combinar datasets de diferentes autores, es habitual encontrar discrepancias en el etiquetado; por ejemplo, un dataset puede etiquetar el balón como

"ball", otro como "football" y otro como "pelota". Roboflow ofrece herramientas de gestión de ontologías que permitieron fusionar y mapear estas etiquetas dispares en una estructura única y coherente, limpiando el ruido de los datos importados de manera sencilla y visual, algo que habría requerido escribir complejos scripts de Python si se hubiera trabajado con herramientas locales.

Sin embargo, la característica técnica que definitivamente validó el uso de Roboflow fue su sistema de anotación inteligente. Como se detallará en apartados posteriores, para lograr una proyección precisa de los jugadores en el tablero 2D, no bastaba con detectarlos mediante un recuadro rectangular (bounding box), ya que esto incluye mucho "ruido" del fondo. Era necesario utilizar Segmentación de Instancia, es decir, dibujar un polígono exacto alrededor de la silueta del jugador. Hacer esto manualmente, punto por punto, para miles de jugadores, es una tarea titánica e ineficiente. Roboflow integra una herramienta denominada Smart Polygons, impulsada por el modelo SAM (Segment Anything Model) (Kirillov et al., 2023). Esta funcionalidad permite que, con un solo clic sobre un jugador, la inteligencia artificial detecte sus bordes y genere una máscara poligonal precisa ajustada al contorno del cuerpo. Esto transformó el flujo de trabajo, permitiendo etiquetar con una precisión de píxel en una fracción del tiempo que tomaría con herramientas tradicionales.

Además, la plataforma permite implementar un ciclo de aprendizaje activo mediante el etiquetado asistido por modelos (Model-Assisted Labeling). Una vez entrenada una primera versión básica del modelo con pocas imágenes, Roboflow permite cargar ese modelo para que pre-etiquete automáticamente las nuevas imágenes que se suben al sistema. De esta forma, el rol del desarrollador pasa de ser un "dibujante" a ser un "supervisor" que solo corrige los fallos de la IA. A medida que el modelo mejoraba, las pre-etiquetas eran cada vez más perfectas, acelerando exponencialmente la creación del dataset final. La herramienta también proporciona estadísticas de salud del dataset (Dataset Health Check), alertando sobre desequilibrios en las clases (por ejemplo, si hay muchas más imágenes de jugadores que de balones) o sobre imágenes duplicadas, permitiendo corregir sesgos antes de iniciar el entrenamiento.

Otro pilar fundamental proporcionado por Roboflow es su motor de Aumento de Datos (Data Augmentation). Dado que no era posible grabar partidos en todas las

condiciones de luz y clima posibles, se utilizó esta funcionalidad para generar datos sintéticos. La herramienta permite multiplicar el tamaño del dataset original aplicando transformaciones aleatorias a las imágenes. Para FutsalVision, se configuró un pipeline de aumento que generaba tres versiones nuevas por cada imagen original (3x), aplicando modificaciones estratégicas: cambios de brillo y exposición para simular entornos oscuros o sobreexpuestos típicos de pabellones municipales y volteo horizontal (horizontal flip) para enseñar al modelo que un jugador corriendo hacia la izquierda es igual que uno corriendo hacia la derecha. Estas variaciones son vitales para evitar el sobreajuste (overfitting) y garantizar que el modelo aprenda características robustas.

Es importante abordar también el aspecto económico y las limitaciones de la herramienta. Roboflow es una plataforma comercial que funciona mediante un sistema de créditos de procesamiento. Sin embargo, gracias al convenio educativo existente, al registrarse con el correo institucional de la Universidad Carlos III de Madrid, se obtuvo acceso al plan para estudiantes e investigadores, el cual otorga una asignación de 30 créditos mensuales. En el contexto de este proyecto, esta cantidad resultó ser suficiente y bien dimensionada. Se consumieron aproximadamente 3 créditos para la generación de las versiones del dataset y el uso de las herramientas de anotación inteligente. Esta disponibilidad de recursos gratuitos de nivel profesional fue una ventaja competitiva clave frente a otras opciones de pago.

No obstante, la herramienta no está exenta de limitaciones, y es crucial mencionar la decisión estratégica tomada respecto al entrenamiento. Aunque Roboflow ofrece la posibilidad de entrenar el modelo directamente en sus servidores con un solo clic ("Train with Roboflow"), se decidió deliberadamente no utilizar esta función. La razón es evitar el "bloqueo del proveedor". Entrenar dentro de la plataforma convierte el modelo en una caja negra alojada en su nube, accesible solo a través de su API y sujeta a sus precios futuros. Para mantener el rigor académico y el control total sobre la ingeniería del sistema, se utilizó Roboflow exclusivamente como gestor y preprocesador de datos. Una vez el dataset estaba limpio, aumentado y particionado correctamente, se utilizó la función de exportación para descargar los datos en formato compatible con YOLOv8.

Tras la exportación del dataset y la decisión estratégica de mantener la independencia tecnológica respecto a plataformas de terceros, el proyecto se adentró en la fase más

exigente a nivel de recursos: el entrenamiento de la red neuronal. Desde el punto de vista de la ingeniería computacional, el entrenamiento de modelos de segmentación profunda como **YOLOv8** implica la ejecución iterativa de millones de operaciones de multiplicación de matrices y cálculo de gradientes en retropropagación. Realizar esta carga de trabajo en una Unidad Central de Procesamiento (**CPU**) convencional es técnicamente inviable; los tiempos de entrenamiento se dilatarían semanas, haciendo imposible la iteración ágil de hiperparámetros necesaria para un Trabajo de Fin de Grado. Por tanto, se hizo imperativo el uso de aceleración por hardware mediante Unidades de Procesamiento Gráfico (**GPU**).

Se evaluó la posibilidad de utilizar infraestructura local, pero se descartó rápidamente por limitaciones físicas y económicas. Entrenar un modelo de estas características en un ordenador portátil estándar provocaría un estrangulamiento térmico (thermal throttling) severo, degradando el rendimiento para proteger el hardware, mientras que adquirir una estación de trabajo con una GPU dedicada de gama alta (como una NVIDIA RTX 3090 o 4090) supondría una barrera de entrada económica injustificable. La alternativa natural en un entorno profesional sería recurrir a la nube pública, utilizando instancias EC2 de **Amazon Web Services (AWS)** o **máquinas virtuales en Microsoft Azure**. Sin embargo, estas plataformas conllevan una carga de trabajo "DevOps" considerable: requieren configurar instancias virtuales, gestionar claves SSH, instalar manualmente los drivers de CUDA y cuDNN, y vigilar estrictamente los costes por hora para evitar facturas inesperadas, desviando así el foco del desarrollo algorítmico hacia el mantenimiento de infraestructura.

Frente a la complejidad de la nube tradicional y la insuficiencia del hardware local, **Google Colab** emergió como la solución óptima para el desarrollo de este MVP. A diferencia de plataformas como **Kaggle Kernels** —que también ofrecen GPUs gratuitas, pero están diseñadas como entornos efímeros para competiciones, donde los datos se borran al cerrar la sesión—, Colab permite una integración nativa y fluida con **Google Drive**. Esta característica fue decisiva, ya que permitió montar un sistema de archivos persistente donde almacenar de forma centralizada los datasets crudos, los pesos de los modelos entrenados (.pt) y los vídeos de prueba.

Desde una perspectiva de hardware, la disponibilidad gratuita de **GPUs NVIDIA Tesla T4** (equipadas con 16GB de memoria VRAM) en Colab marcó la diferencia técnica. La cantidad de VRAM es un factor crítico en la segmentación de instancias, ya que determina el tamaño del lote (batch size) máximo que se puede procesar simultáneamente. Una GPU local de gama media, con 6 u 8 GB de VRAM, obligaría a utilizar tamaños de lote muy reducidos, lo que introduce ruido en la estimación del gradiente y desestabiliza la convergencia del modelo. Los 16GB de la Tesla T4 permitieron configurar batch sizes robustos, garantizando un aprendizaje más estable y rápido. Además, el entorno de Colab elimina la fricción de configuración del software, conocida coloquialmente como el "infierno de dependencias" (dependency hell). Las bibliotecas esenciales para el proyecto ya vienen preinstaladas y optimizadas: PyTorch como motor de tensores, Ultralytics para la arquitectura YOLO, OpenCV para las operaciones de visión y homografía, y NumPy para el cálculo matricial, permitiendo que el código fuente escrito en Python 3.10 fuera ejecutable de inmediato.

Finalmente, es importante realizar un ejercicio de honestidad ingenieril respecto a la arquitectura elegida. Si bien la combinación monolítica de Google Colab, Drive y scripts en Python es la configuración ideal para la validación rápida de hipótesis y el desarrollo de un prototipo funcional, se es consciente de sus limitaciones para un despliegue comercial a gran escala. En un escenario de producción futuro, esta arquitectura debería evolucionar hacia un sistema de **microservicios desacoplados**: sustituyendo Google Drive por almacenamiento de objetos escalable (como **AWS S3**) para la gestión de vídeos masivos, implementando una base de datos relacional (como **PostgreSQL**) para la gestión de usuarios y partidos, y "dockerizando" el modelo de inferencia para desplegarlo en clústeres de **Kubernetes** que permitan autoescalado bajo demanda.

En conclusión, la arquitectura de *FutsalVision* busca el equilibrio perfecto entre potencia y agilidad. Todo el desarrollo se fundamenta en **Python 3.10** y el uso de **YOLOv8**, elegido por ser el líder actual en detección rápida y ligera. Para optimizar el tiempo, he centralizado la gestión de datos en **Roboflow**, automatizando las tareas manuales de etiquetado, mientras que la potencia bruta de cálculo la he obtenido de las **GPUs Tesla T4 de Google Colab**. Esta combinación me ha permitido desplegar un flujo de trabajo de alto rendimiento sin depender de servidores físicos costosos, maximizando la velocidad de desarrollo para cumplir con los objetivos del sistema.

3. Análisis y Diseño del Sistema

3.1 Definición del problema y dificultades

El punto de partida para el diseño de FutsalVision requiere una comprensión profunda de la asimetría existente entre la masividad de la práctica deportiva y la escasez de herramientas tecnológicas aplicadas a ella. El fútbol sala en España no es un deporte minoritario; las cifras oficiales de la Real Federación Española de Fútbol recogen más de 156.000 licencias federativas (Real Federación de Fútbol de Madrid [RFFM], s.f.), un dato que apenas representa la punta del iceberg de la realidad social, pues excluye el vasto ecosistema de competiciones escolares, ligas de empresas y torneos municipales que movilizan a cientos de miles de jugadores semanalmente. Sin embargo, existe una desconexión tecnológica evidente. Mientras que disciplinas como el tenis han normalizado el uso del "Ojo de Halcón" o el fútbol 11 profesional depende del VAR y la telemetría avanzada para la toma de decisiones, el fútbol sala amateur permanece anclado en metodologías analógicas. Si bien la barrera de la grabación se ha superado gracias a los teléfonos móviles con cámaras de alta definición, el procesamiento de esas imágenes sigue siendo un cuello de botella crítico: el análisis es manual, subjetivo y extremadamente ineficiente en tiempo, requiriendo que un entrenador dedique horas a visionar y cortar clips "a ojo", una tarea inviable para cuerpos técnicos no profesionales.

La persistencia de este problema no se debe a una falta de interés del mercado, sino a la existencia de barreras técnicas y ambientales que hacen que la automatización del videoanálisis en este deporte sea un desafío. La primera y más evidente dificultad radica en la alta variabilidad del dominio visual. A diferencia del fútbol 11, donde el algoritmo puede asumir con seguridad que el terreno de juego será siempre una superficie verde uniforme con líneas blancas contrastadas, el fútbol sala presenta un escenario caótico para una red neuronal. Los pabellones municipales y pistas escolares carecen de estandarización: el suelo puede ser parqué de madera con altos índices de reflexión especular, goma azul mate, cemento gris o superficies sintéticas de colores vivos. A esta heterogeneidad cromática se suma el problema del "ruido estructural": la inmensa mayoría de las pistas son polideportivas, lo que implica que sobre el terreno de juego conviven líneas de baloncesto, voleibol, balonmano e incluso tenis. Para un sistema de

detección de líneas, discriminar cuáles pertenecen a los límites del campo de fútbol sala y cuáles son interferencias de otros deportes supone un reto de segmentación semántica complejo que no existe en estadios profesionales dedicados.

El segundo obstáculo crítico es la inestabilidad de la captura de imagen y la perspectiva. En un entorno de retransmisión profesional, las cámaras son fijas, están situadas en puntos elevados estratégicos y cubren ángulos panorámicos predecibles. En el contexto amateur al que se dirige este proyecto, la grabación se realiza habitualmente "a pie de pista" o desde gradas bajas, con un dispositivo móvil sostenido a mano por un familiar o un técnico. Esto introduce dos problemas algorítmicos severos: la ausencia de una visión panorámica completa (el campo de visión suele cubrir apenas un 20% o 30% de la pista) y la variabilidad constante de los parámetros extrínsecos de la cámara. Al tener que realizar movimientos constantes seguimiento para mantener el balón en encuadre, el sistema no puede confiar en una matriz de homografía estática; debe recalcular la transformación de perspectiva en cada fotograma para proyectar correctamente a los jugadores en el tablero 2D, lidiando además con posibles oclusiones causadas por redes de protección, barandillas o el propio público que se cruza por delante de la lente.

A estas dificultades geométricas se suman las condiciones de iluminación no controladas. Al tratarse de vídeos grabados tanto en pabellones cerrados como en pistas exteriores, el algoritmo debe ser robusto frente a cambios drásticos de luz. En interiores, es frecuente encontrar iluminación artificial antigua que provoca parpadeo en la grabación o zonas de sombra profunda en las esquinas; además, los suelos de madera barnizada actúan a menudo como espejos, generando reflejos de los focos o de las ventanas que pueden ser falsamente identificados como objetos o líneas por un detector no entrenado. En exteriores, la posición del sol o las sombras proyectadas por edificios adyacentes alteran la saturación y el contraste de la imagen, impidiendo el uso de técnicas clásicas de visión basadas en umbrales de color (color thresholding) para la detección de camisetas o balón.

Finalmente, la propia dinámica del juego y las limitaciones de infraestructura añaden una capa extra de complejidad. El fútbol sala es un deporte de alta densidad en espacio reducido, caracterizado por un juego de contacto, bloqueos constantes y movimientos

explosivos. Esto provoca situaciones frecuentes de oclusión severa, donde los jugadores se agrupan en "barullos" que impiden ver al sujeto completo, dificultando el seguimiento individual (tracking) y la re-identificación de identidades tras un cruce. Asimismo, la normativa de cambios volantes —donde las sustituciones se realizan con el balón en juego y sin detener el reloj— implica que los jugadores entran y salen del encuadre de forma impredecible, a menudo por zonas periféricas que la cámara no está enfocando. Todo esto ocurre en un contexto donde la instalación de hardware fijo (cámaras tipo Veo o sistemas de antenas) es inviable administrativamente, ya que las pistas son compartidas por múltiples clubes y ayuntamientos, obligando a que la solución sea puramente basada en software y capaz de inferir la táctica a partir de una grabación monocular imperfecta, parcial y ruidosa

3.2 Requisitos del sistema

Para dar respuesta a la problemática planteada y superar las barreras técnicas del entorno de fútbol sala amateur, se han definido los siguientes requisitos que rigen el diseño y la implementación de FutsalVision.

3.2.1. Requisitos Funcionales (RF)

Los requisitos funcionales describen los comportamientos específicos y las funciones que el sistema debe ser capaz de ejecutar para cumplir con su objetivo de análisis táctico automatizado.

- **RF-01. Ingesta y Decodificación de Vídeo Monocular:** El sistema debe ser capaz de cargar y procesar archivos de vídeo estándar (formatos .mp4, .mov, .avi) provenientes de dispositivos móviles no profesionales. Debe permitir la lectura secuencial de fotogramas (frames) sin depender de metadatos de sensores externos ni de hardware de captura propietario.
- **RF-02. Detección Multi-Clase mediante Segmentación de Instancia:** El sistema debe identificar y clasificar automáticamente los elementos relevantes del juego en cada fotograma. A diferencia de la detección por cajas (bounding boxes), el sistema debe generar máscaras de segmentación precisas (polígonos) para las siguientes clases:

- Jugadores: Distinguiendo la silueta para calcular el centroide de la base (pies).
- Balón: Detectando el objeto esférico incluso en condiciones de movimiento rápido (motion blur).
- Árbitros y outsiders como entrenadores y jugadores de banquillo: Para excluirlos del análisis táctico de los equipos.

- **RF-03. Reconocimiento de Puntos Clave del Terreno:** El algoritmo debe detectar automáticamente características visuales estáticas de la pista (terreno de juego, áreas, porterías, córners, círculo central) necesarias para la calibración geométrica, discriminando las líneas de fútbol sala de las de otros deportes (baloncesto, voleibol) presentes en el pabellón.
- **RF-04. Estimación de FOV:** Dado que la cámara puede realizar movimientos de rotación y traslación para seguir la jugada, el sistema debe implementar un mecanismo de registro espacial que identifique qué sección del mapa global está siendo cubierta por el cono de visión del sensor en cada instante.
- **RF-05. Proyección Inversa de Perspectiva (Mapping 2D):** El sistema debe ejecutar una transformación matemática de dominio, traduciendo las coordenadas de píxeles (u, v) de la imagen original —las cuales están sujetas a distorsión perspectiva y escala variable— a un sistema de coordenadas métricas normalizadas (x, y) sobre un tablero virtual, usando diversas funciones de transformación según el ángulo de la cámara.
- **RF-06. Generación de Vídeo Sintético Táctico**: El sistema debe compilar la secuencia de coordenadas procesadas para renderizar y exportar un archivo de vídeo. En esta visualización, el entorno real se sustituye por una plantilla gráfica estandarizada de un campo de fútbol sala, sobre la cual se representan los elementos detectados: los jugadores deben aparecer como puntos o iconos de color sólido (diferenciados por equipos) y el balón como un elemento gráfico distintivo de menor tamaño. El resultado debe mostrar el movimiento fluido de estos elementos en tiempo real, eliminando cualquier distracción visual del fondo original, mostrando en este tablero los elementos que se observan en el video proporcionado por el usuario.
- **RF-07. Extracción del Estado Inicial (Frame Zero)**: El sistema debe generar una representación estática correspondiente al primer instante del análisis ($t=0$) para establecer la situación táctica base. Dado el alcance del MVP y el uso de una

única cámara monocular, el sistema priorizará la ubicación precisa de los jugadores y el balón que se encuentren dentro del Campo de Visión (FOV) efectivo. Para los elementos que queden fuera del encuadre inicial, el sistema gestionará su ausencia mediante la lógica de inicialización, limitándose a representar la información visualmente verificable o inferida por reglas de contexto básicas, sin inventar posiciones no observadas.

3.2.2. Requisitos No Funcionales (RNF)

Los requisitos no funcionales definen las restricciones técnicas, el entorno operativo y los estándares de calidad bajo los cuales debe operar el sistema.

- **RNF-01. Pila Tecnológica y Lenguaje:** El núcleo del sistema debe estar desarrollado íntegramente en Python (versión 3.10 o superior), aprovechando su ecosistema de librerías para ciencia de datos. Se utilizarán OpenCV para el procesamiento de imagen y álgebra computacional y Ultralytics para la implementación de la arquitectura de red neuronal.
- **RNF-02. Arquitectura de Deep Learning:** El modelo de detección debe basarse en la arquitectura YOLOv8 (versión Segmentación) debido a su equilibrio óptimo entre velocidad de inferencia y precisión (mean Average Precision), descartando arquitecturas más pesadas como Mask R-CNN para facilitar la futura portabilidad.
- **RNF-03. Entorno de Ejecución y Hardware:** El sistema está diseñado para ejecutarse en un entorno de computación en la nube, específicamente en Google Colab, aprovechando la aceleración por hardware mediante GPUs NVIDIA Tesla T4 (o superior) con al menos 16 GB de VRAM para soportar el entrenamiento y la inferencia de modelos de segmentación profunda.
- **RNF-04. Gestión del Ciclo de Vida del Dato:** La gestión, versionado, preprocesamiento y aumento de datos (Data Augmentation) debe realizarse a través de la plataforma Roboflow, garantizando la trazabilidad del dataset y la unificación de ontologías de clases antes de la fase de entrenamiento.
- **RNF-05. Persistencia y Almacenamiento:** El sistema debe integrarse con Google Drive como sistema de archivos persistente para el almacenamiento de los datasets de entrenamiento, los pesos de los modelos (.pt) y los resultados generados, mitigando la volatilidad del entorno de ejecución de Colab.
- **RNF-06. Independencia del Hardware de Captura:** El algoritmo no debe depender de una calibración intrínseca previa de una cámara específica. Debe ser

robusto frente a vídeos grabados con diferentes dispositivos móviles, distancias focales y resoluciones, siempre que se garantice un mínimo de visibilidad de la pista.

- **RNF-07. Adaptabilidad Ambiental y Generalización:** El modelo debe demostrar capacidad de generalización ante variaciones lumínicas y cromáticas estándar propias de las pistas no profesionales. Para escenarios con desviaciones visuales significativas (cambios drásticos en el color del pavimento o condiciones de iluminación extremas), el sistema debe estar diseñado para permitir una adaptación rápida mediante técnicas de *Fine-Tuning* o *Few-Shot Learning*, garantizando la recuperación de la funcionalidad operativa con un conjunto reducido de nuevas muestras de calibración, sin requerir la reconstrucción total del dataset.

3.3 Cómo se interactúa con el sistema

El diseño de la interacción de *FutsalVision* se ha abordado desde dos perspectivas complementarias: una visión conceptual centrada en la experiencia de usuario (UX) y una formalización técnica mediante estándares de ingeniería del software.

3.3.1. Experiencia de Usuario: El concepto "Caja Negra"

Para diseñar cómo interactúa el usuario con FutsalVision, la prioridad ha sido la simplicidad: queremos que el entrenador se preocupe de grabar y analizar, no de programar. Por eso, hemos definido un flujo de trabajo donde el humano solo interviene al principio y al final, dejando que el sistema haga todo el trabajo pesado en la nube.

En el funcionamiento normal (el "camino feliz"), el proceso es muy lineal. El usuario empieza grabando el partido con su móvil. Después, sube ese archivo de vídeo a una carpeta de Google Drive y ejecuta el sistema. Aquí es donde termina su trabajo manual: el sistema se encarga de coger ese vídeo, procesarlo y devolverle el análisis listo en la misma carpeta.

El siguiente esquema resume este camino sencillo, desde que se graba hasta que se recibe el resultado:

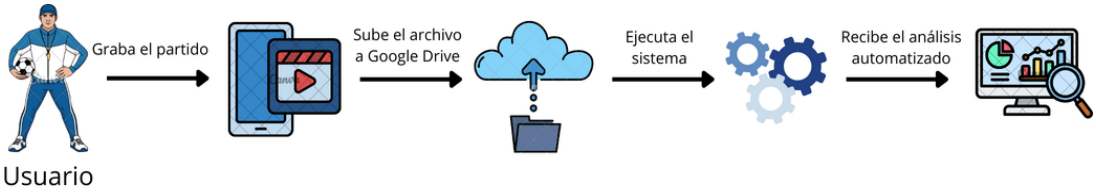

Ilustración 1. Diagrama del Usuario

Ahora bien, ¿qué pasa dentro del sistema mientras el usuario espera? Para el usuario, el software funciona como una "caja negra": sabe lo que entra (el vídeo del partido) y sabe lo que sale (el vídeo táctico), pero no necesita ver los cálculos internos.

Internamente, cuando el sistema recibe la orden de empezar, activa su motor de procesamiento. Aquí es donde ocurre la "magia" de la Inteligencia Artificial: por un lado, se detectan los objetos (jugadores y balón) y, por otro, se calcula la geometría de la pista para saber desde dónde estamos mirando. Todo esto se combina para transformar la imagen y generar el vídeo final, tal y como se ve en este diagrama conceptual:

Ilustración 2. Diagrama idílico del sistema

Sin embargo, hay que ser realistas con el estado actual del proyecto. Al tratarse de un MVP (Producto Mínimo Viable) y dado que cada pista de fútbol sala es un mundo diferente (distintos colores de suelo, iluminación, líneas de otros deportes), el sistema a veces necesita una pequeña ayuda para "aprender" a ver una pista nueva.

Si el sistema se encuentra con un pabellón que nunca ha visto, el flujo automático no es suficiente. En estos casos, usamos una estrategia de "Human-in-the-loop" (humano en el bucle). El proceso es el siguiente: extraemos unas pocas imágenes del vídeo (unas 25), las etiquetamos manualmente para enseñarle al modelo cómo es esa pista en concreto

y reentrenamos la IA rápidamente. Una vez el modelo ha aprendido las particularidades de ese nuevo campo, ya puede procesar el resto del partido automáticamente.

Este paso extra, que es clave para garantizar que el análisis sea preciso en este prototipo, se detalla en el último esquema:

Ilustración 3. Diagrama real del sistema

3.3.2. Formalización Técnica: Diagrama de Casos de Uso (UML)

Una vez definido el flujo conceptual desde la perspectiva del usuario, es necesario traducir dicha experiencia en especificaciones técnicas inequívocas bajo los estándares de la Ingeniería del Software. Para ello, se ha modelado el comportamiento del sistema utilizando el lenguaje unificado de modelado (UML), concretamente mediante un **Diagrama de Casos de Uso.**

Esta representación formal nos permite definir con precisión los límites del sistema y las responsabilidades. A diferencia del modelo conceptual simplificado, el modelo técnico identifica dos actores diferenciados:

1. **Analista Deportivo (Usuario Final**): Consume el servicio de análisis.
2. **Administrador del Sistema (Equipo Técnico):** Interviene únicamente en tareas de mantenimiento y adaptación del modelo (soporte).

El siguiente diagrama ilustra la arquitectura funcional completa y la separación de roles:

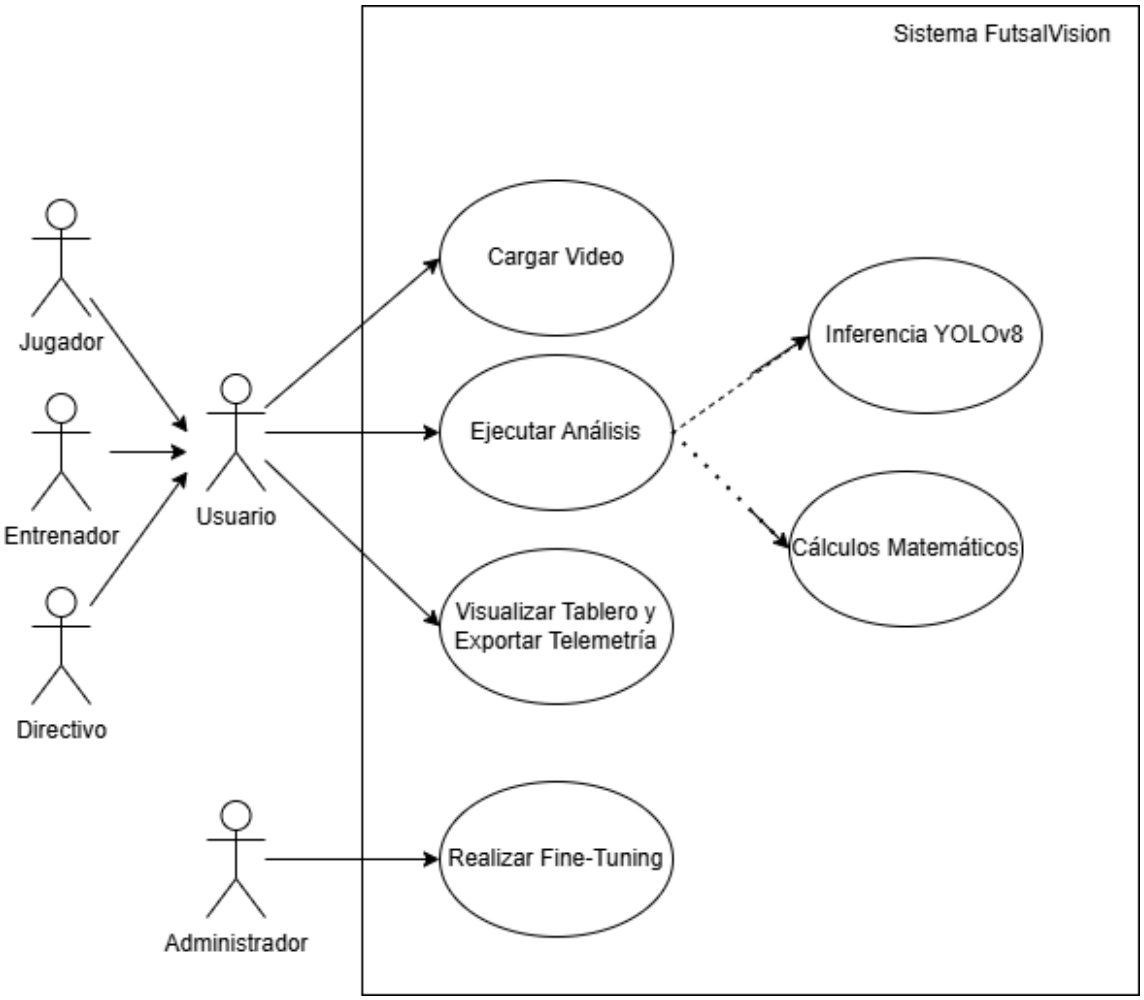

Ilustración 4. Diagrama de Casos

A. Interacción del Usuario

1. **Cargar Vídeo del Partido:** Constituye el punto de entrada. El actor aloja el material audiovisual (archivos .mp4 o .mov) en el directorio designado. A nivel técnico, el sistema valida el formato y la integridad del archivo en la nube.
2. **Ejecutar Pipeline de Análisis:** Es el caso de uso central. Al ser invocado, orquesta automáticamente dos sub-procesos transparentes (relaciones <<include>>):
   - *Inferencia YOLOv8*: Detección y segmentación frame a frame.
   - *Cálculos Matemáticos*: Transformación geométrica al plano 2D.
3. **Visualizar y Exportar Telemetría**: Fase de validación y persistencia. Permite verificar el tablero táctico y generar los archivos estructurados (.csv) para su uso en herramientas de Business Intelligence.

B. Interacción del Administrador del Sistema

4. **Realizar Fine-Tuning (Mantenimiento Adaptativo):** Este caso de uso da respuesta técnica al requisito de Adaptabilidad (RNF-07) sin cargar de complejidad al usuario final. Cuando se detecta que el rendimiento del modelo decrece en un nuevo pabellón (debido a cambios drásticos en iluminación o color de pista), el Administrador interviene manualmente. Este proceso implica seleccionar un subconjunto de imágenes críticas, etiquetarlas y ejecutar un reentrenamiento parcial de las últimas capas de la red. De esta forma, el sistema evoluciona y se adapta a nuevos escenarios de forma transparente para el entrenador.

3.4 Arquitectura de Componentes

En el apartado anterior hemos visto cómo el sistema actúa como una "caja negra" para el usuario final: entra un vídeo y sale un análisis. Sin embargo, desde el punto de vista de este TFG, esa caja no es un bloque único, sino un engranaje de piezas de software que trabajan en cadena.

Para diseñar FutsalVision, hemos huido de crear un código monolítico (un script gigante imposible de mantener). En su lugar, hemos optado por una arquitectura modular basada en componentes desacoplados. Esto significa que cada parte del código se encarga de una única tarea muy específica y le pasa el relevo a la siguiente.

El siguiente diagrama UML muestra "qué hay dentro de la caja negra" y cómo fluye la información paso a paso dentro del entorno de Google Colab:

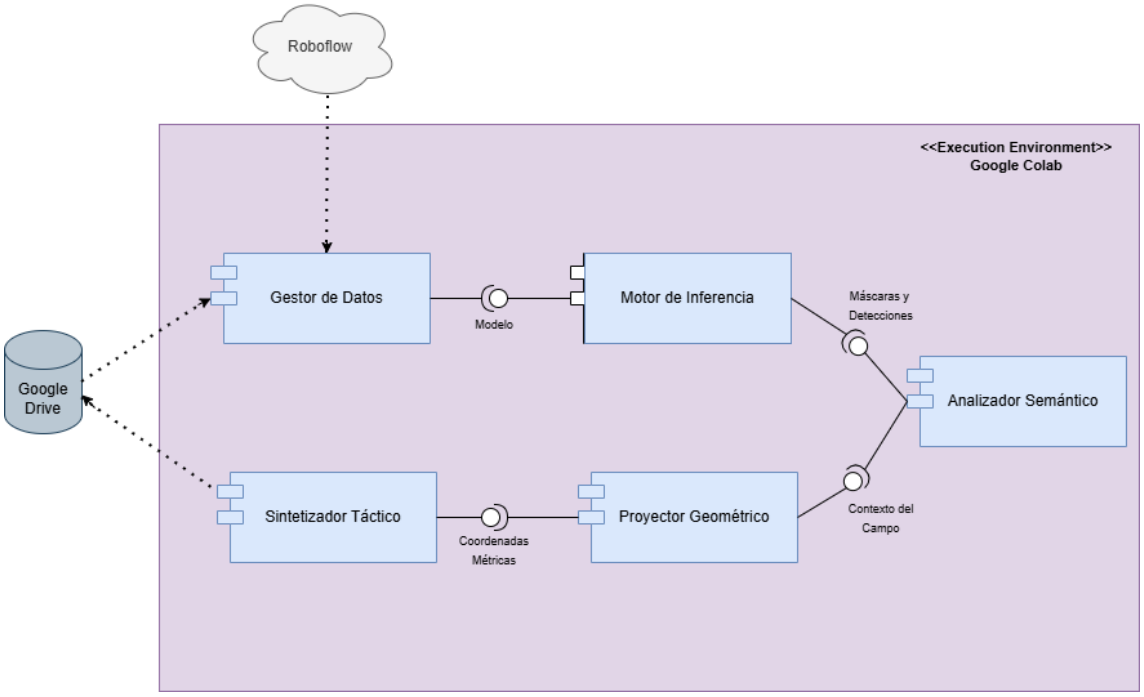

Ilustración 5. Diagrama UML

Como se observa en el esquema, el sistema se divide en cinco módulos secuenciales. A continuación, explicamos qué responsabilidad tiene cada uno:

1. **Gestor de Datos (Data Ops):** Este componente constituye el punto de partida de toda la arquitectura, actuando como el administrador central del conocimiento del sistema. Su responsabilidad principal es orquestar la comunicación entre el almacenamiento persistente (Google Drive) y la plataforma de etiquetado (Roboflow) para suministrar el archivo de pesos del modelo (.pt) actualizado. Esta etapa es crítica para la viabilidad del proyecto: al ser la fuente primaria de información, es el módulo encargado de asegurar la calidad de las detecciones mediante el reentrenamiento continuo. Si este gestor no provee un modelo preciso, el error se propagaría en cascada al resto de etapas, invalidando el posterior análisis geométrico y táctico.
2. **Motor de Inferencia (La Visión):** Una vez cargado el modelo con los pesos óptimos, este componente ejecuta el procesamiento visual del flujo de vídeo. Utilizando la arquitectura de red neuronal YOLOv8, realiza la detección y segmentación de instancias fotograma a fotograma. Desde una perspectiva técnica, este módulo opera en un nivel de abstracción puramente visual: su función se limita a la identificación de patrones y la generación de máscaras binarias y

siluetas de los objetos de interés. Entrega la información geométrica cruda (ubicación en píxeles) sin atribuirle aún un contexto espacial o táctico, delegando la interpretación semántica a los módulos subsiguientes.

3. **Analizador Semántico (El Contexto):** Aquí es donde empieza la inteligencia lógica. Este módulo recibe las detecciones y les da sentido. Es el encargado de diferenciar si estamos viendo el "Campo 1" o el "Campo 2", y de identificar qué portería es la local y cuál la visitante. Sin este contexto, no podríamos situar a los jugadores en el mapa correctamente.
4. **Proyector Geométrico (Las Matemáticas):** Es el núcleo matemático del TFG. Recibe el contexto del campo y calcula dónde está la cámara. Dependiendo de lo que se vea en pantalla, elige dinámicamente entre tres fórmulas matemáticas distintas (basadas en líneas de banda, áreas o mixtas) para transformar los píxeles de la imagen en coordenadas métricas reales (metros). Básicamente, traduce "píxeles de pantalla" a "posiciones en el suelo".
5. **Sintetizador Táctico (El Resultado):** Como etapa final de la cadena de procesamiento, este módulo consolida los datos transformados para generar los entregables visuales y asegurar la coherencia deportiva del análisis. Su salida es dual: por un lado, genera y exporta una imagen estática del "Frame 0" o estado inicial, garantizando que aparezcan los 10 jugadores posicionados para el análisis de la formación de salida. Por otro lado, renderiza la secuencia de vídeo completa, compilando frame a frame el movimiento de todos los elementos detectados sobre el tablero 2D. Este vídeo final permite visualizar el flujo dinámico del partido y las trayectorias en tiempo real, guardando ambos activos (imagen y vídeo) en Google Drive para su disposición final.

4. Implementación Técnica

En este capítulo se detalla el desarrollo del sistema, las decisiones de diseño adoptadas y la evolución del modelo a través de las distintas iteraciones experimentales.

4.1. Gestor de Datos

El primer y más crítico componente del sistema es la gestión de los datos. El objetivo principal de este módulo es la creación de un modelo de visión artificial capaz de generalizar, es decir, detectar elementos en cualquier vídeo de fútbol sala independientemente del color de la pista, la iluminación o el ángulo de cámara.

4.1.1. Adquisición y Unificación de Datasets Heterogéneos

Para nutrir el sistema, la estrategia inicial consistió en recopilar datasets de código abierto existentes para evitar comenzar el etiquetado desde cero. Se utilizó la plataforma Roboflow, creando un proyecto unificado que consolidó aproximadamente 10 datasets distintos, sumando un total inicial de 4.971 imágenes. Esto proporcionó una gran variedad de contextos visuales: pistas de parqué, goma azul o cemento, así como grabaciones desde ángulos cenitales, córners y a pie de pista.

Sin embargo, la fusión de estas fuentes presentó dos problemas estructurales graves que impedían su uso directo:

1. **Inconsistencia Taxonómica:** Cada dataset utilizaba nomenclaturas distintas (ej. "player" vs "jugador"), y muchos carecían de distinción entre equipos o ignoraban clases estructurales vitales para la táctica (como las líneas del área).
2. **Inadecuación de los Bounding Boxes**: La gran mayoría de los datos venían etiquetados con Object Detection (cajas rectangulares).

Para abordar el problema de la geometría, se analizó qué tipo de técnica de visión artificial era la más adecuada. Roboflow y la arquitectura YOLO permiten cuatro enfoques principales, de los cuales se seleccionó el más apto:

- **Object Detection (Bounding Box):** Devuelve las coordenadas de un rectángulo (x_centro, y_centro, ancho, alto).
- **Instance Segmentation**: Detecta objetos mediante polígonos que se ajustan al contorno preciso [clase, x1, y1, x2, y2, ..., xn, yn].
- **Semantic Segmentation:** Clasifica cada píxel de la imagen con una etiqueta, pero no distingue entre instancias individuales (todos los jugadores serían una mancha única de "persona").

- **Keypoint Detection:** Detecta puntos de interés (esqueletos, articulaciones). Aunque prometedor para análisis biomecánico, se consideró una fase posterior, priorizando primero la ubicación espacial precisa mediante segmentación.

Ilustración 6. Opción A, Bounding Boxes

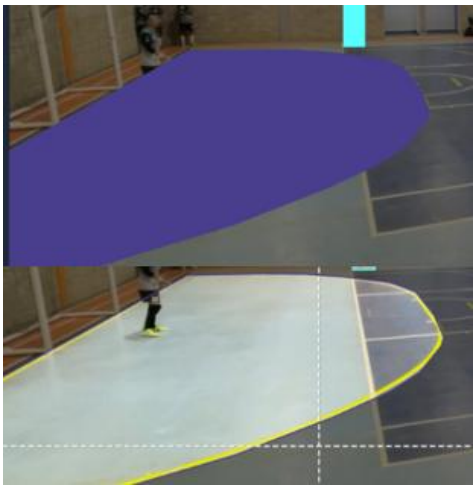

Ilustración 7. Opción B, Instance Segmentation

Se descartó el uso de Bounding Boxes (Opción A en la figura anterior donde se identifica un área) porque los rectángulos no capturan la geometría real de una perspectiva deformada por la cámara. Para realizar la transformación a un plano 2D, el sistema necesita conocer los vértices exactos de la pista y la posición precisa de los pies de los jugadores, no un área aproximada. Por ello, se optó por Instance Segmentation, que define polígonos ajustados al contorno del objeto.

4.1.2. Definición de Clases y Refinamiento del Etiquetado

Para resolver la inconsistencia, se diseñó una taxonomía maestra de 12 clases y se procedió a un re-etiquetado masivo. Las clases seleccionadas fueron: ball, corner, court (pista), goal (portería), midfield (medio campo), area, referee, outsider (suplentes/público/entrenadores) y las distinciones de equipo (goalkeeper-team1, player-team1, etc.).

Dado que ningún dataset público cumplía con estos requisitos de segmentación y clases específicas, se procedió a un re-etiquetado manual laborioso (5-10 minutos por imagen). El flujo de trabajo se optimizó mediante la herramienta "Smart Polygon" de Roboflow. Esta herramienta funciona mediante un modelo de asistencia interactiva

(similar a Segment Anything): el usuario dibuja un recuadro aproximado alrededor del objeto y la herramienta infiere automáticamente los bordes del objeto basándose en el contraste y gradientes de color, generando un polígono ajustado. Esto transformó las antiguas bounding boxes en segmentaciones precisas.

Durante esta fase, se detectó un problema crítico con la clase midfield. Inicialmente definida solo como el círculo central, el modelo la confundía con otros círculos que estaban en el campo como el de baloncesto o logos de patrocinadores. Como solución se redefinió la clase midfield para englobar no solo el círculo, sino también la línea divisoria central completa. Esta decisión no solo mejoró la detección, sino que se convirtió en la clave algorítmica para que, posteriormente, el código pueda determinar qué lado es "izquierda" y qué lado es "derecha" al dividir el polígono de la pista.

4.1.3. Aprendizaje Asistido y Adaptación de Dominio

Una vez definida la taxonomía de clases, se procedió a realizar una primera prueba de concepto para validar la viabilidad del sistema. Para ello, se entrenó un modelo preliminar utilizando exclusivamente las primeras 250 imágenes etiquetadas, las cuales pertenecían a un único contexto visual (un partido específico con una iluminación y color de pista determinados).

Los resultados de este primer experimento revelaron una limitación crítica. Si bien el modelo demostraba un rendimiento excelente sobre el vídeo original, al intentar realizar inferencias sobre partidos nuevos —con diferentes colores de equipación, pistas de parqué en lugar de sintético o distinta iluminación—, la capacidad de detección descendía drásticamente. El sistema sufría de un claro sobreajuste (overfitting) al contexto inicial: había aprendido a detectar "jugadores de camiseta roja en pista azul", pero no había abstraído el concepto general de "jugador de fútbol sala".

Ante esta situación, se planteó el desafío de cómo escalar el dataset. La perspectiva de etiquetar manualmente cientos de imágenes para cada nuevo tipo de pista resultaba inviable en términos de tiempo y esfuerzo. Fue en este punto donde se decidió integrar técnicas de "Human-in-the-loop" mediante la herramienta Label Assist de Roboflow.

Esta decisión transformó radicalmente el flujo de trabajo. En lugar de comenzar el etiquetado de nuevas imágenes desde cero (un "lienzo en blanco"), se utilizó el modelo preliminar —aunque imperfecto— para realizar una pre-anotación automática de los nuevos datos. El cambio fue sustancial: la tarea pasó de ser una creación manual de polígonos a ser una labor de supervisión y corrección. El modelo pre-entrenado, aunque fallaba en la geometría exacta de la pista (generando a veces polígonos irregulares para la clase court o area), demostró ser sorprendentemente eficaz localizando a los jugadores y a los outsiders. Esto permitió reducir el tiempo de etiquetado por imagen significativamente, ya que el operador humano solo debía ajustar los vértices de la pista y corregir confusiones puntuales entre equipos.

Gracias a esta aceleración en el proceso de etiquetado, fue posible diseñar un experimento empírico para resolver la duda principal de este MVP: ¿Cuánta información necesita el sistema para "entender" un vídeo nuevo?

Se llevó a cabo un proceso iterativo de adaptación de dominio. Se tomó un modelo base entrenado con una gran variedad de pistas (aprox. 500 imágenes) y se intentó inferir sobre un vídeo totalmente desconocido. Como era de esperar, los resultados iniciales fueron pobres. A continuación, se fueron añadiendo progresivamente imágenes de este nuevo vídeo al conjunto de entrenamiento:

1. **Con 5 imágenes nuevas**: El modelo comenzó a detectar la pista, pero seguía confundiendo a los jugadores si se agrupaban o si cambiaban de dirección, y fallaba en las áreas de sombra.
2. **Con 15 imágenes nuevas:** La detección de la infraestructura (court, goal) se estabilizó, pero persistían errores en la clasificación de los equipos (confusión entre player-team1 y player-team2).
3. **Con 25 imágenes nuevas**: Se alcanzó el punto de equilibrio óptimo.

Se determinó que un conjunto de 25 imágenes representativas es el umbral mínimo necesario para que el modelo realice una transferencia de aprendizaje efectiva. Estas 25 imágenes no se seleccionan al azar, sino que deben cubrir una variedad estratégica de situaciones: diferentes zonas del campo, agrupaciones de jugadores (para que el modelo aprenda a separarlos), y momentos de distinta iluminación. Con este pequeño aporte de datos ("Fine-tuning"), lo cual significa que no empieza de cero si no que coge el modelo

anterior y añade estas últimas imágenes, el modelo es capaz de aprender los rasgos específicos del nuevo entorno —como el tono exacto de las camisetas o el brillo del suelo— y generalizar correctamente sobre los miles de frames restantes del vídeo.

4.1.4. Preprocesamiento y Aumento de Datos (Data Augmentation)

Antes de exportar los datos al entorno de Google Colab, se aplicaron técnicas de preprocesamiento para estandarizar la entrada a la red neuronal:

- **Auto-Orient:** Normalización de la orientación basada en metadatos EXIF para evitar que imágenes tomadas con móviles en vertical se procesen rotadas.
- **Resize (Redimensionado):** Todas las imágenes se ajustaron a 640x640 píxeles. Este es el tamaño de entrada nativo óptimo para la arquitectura YOLOv8, equilibrando velocidad de inferencia y precisión.

Para combatir la escasez de datos (525 imágenes originales) y mejorar la robustez, se implementó un pipeline de Data Augmentation (aumento de datos sintético), generando variaciones de cada imagen original. Se seleccionaron transformaciones que simulan condiciones reales de grabación:

1. **Flip Horizontal:** Simula el efecto espejo (jugadores corriendo en dirección opuesta), duplicando la variabilidad direccional.
2. **Rotation (±10º):** Simula imperfecciones en la nivelación del trípode o cámara en mano.
3. **Brightness / Exposure (±15%):** Vital para que el modelo sea invariante a cambios de luz (partidos de día vs. noche, sombras en la pista).
4. **Blur (Desenfocado):** Simula el desenfoque de movimiento (motion blur) típico en deportes rápidos o cámaras de baja calidad.

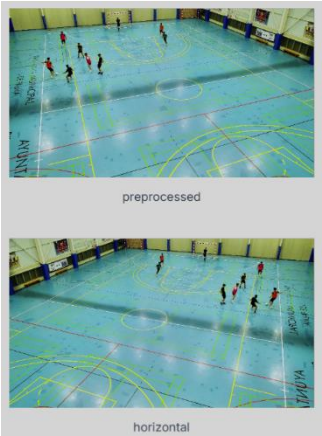

Ilustración 8. Flip Horizontal

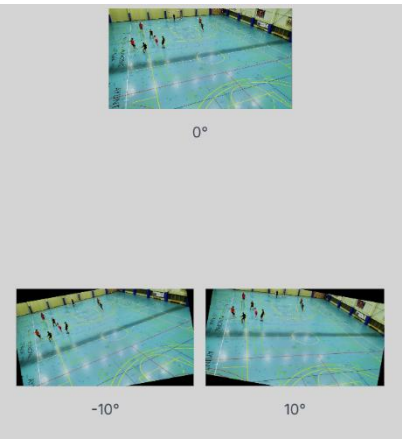

Ilustración 11. Rotation

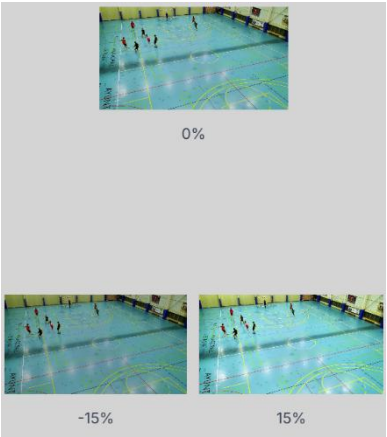

Ilustración 10.Brightness / Exposure

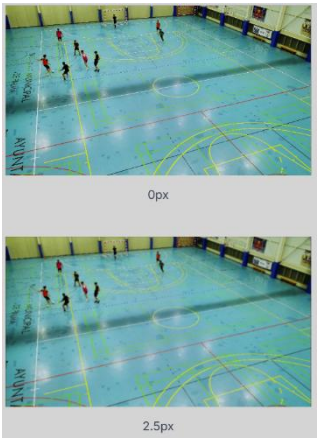

Ilustración 9. Blur

4.1.5. Configuración del Entrenamiento y División del Dataset

Finalmente, el dataset de 1.259 imágenes resultantes se dividió siguiendo la proporción 70% Train, 20% Validation, 10% Test.

- **Train (70%):** Datos que el modelo "ve" para ajustar sus pesos.
- **Validation (20%):** Datos usados durante el entrenamiento para evaluar el rendimiento tras cada época. Es crucial para ajustar los hiperparámetros y detener el entrenamiento si el modelo deja de mejorar, evitando el memorizado de datos (overfitting).
- **Test (10%):** Datos nunca vistos, usados solo para la métrica final de rendimiento.

El entrenamiento se ejecutó en Google Colab utilizando una GPU Tesla T4. Se empleó el script Modelo1.ipynb configurado con los siguientes hiperparámetros clave:

- **Epochs = 100:** Se determinó que 100 ciclos son suficientes para que la función de pérdida se estabilice sin desperdiciar recursos de cómputo (GPU).
- **Batch Size = 16:** Seleccionado para maximizar el uso de la VRAM de la GPU sin causar desbordamientos de memoria.
- **Arquitectura**: YOLOv8-Nano-Segmentation. Se eligió la versión "Nano" por ser la más ligera, permitiendo inferencia en tiempo real sin sacrificar excesiva precisión en las formas grandes como la pista (court).

4.1.6. Definición del Caso de Uso Real (MVP)

Para validar la capacidad de adaptación del sistema en un entorno de producción real, se seleccionó un vídeo aleatorio de internet, externo a los datasets de entrenamiento originales. Este vídeo presenta condiciones de iluminación y color de pista nunca vistas por el modelo base, sirviendo como la prueba definitiva para el Producto Mínimo Viable (MVP).

El clip seleccionado tiene una duración de 28 segundos, lo que se traduce en un total de 737 frames (a 30 fps). Etiquetar la totalidad de estos frames sería ineficiente y contrario a la filosofía de automatización del proyecto. Aplicando la regla empírica definida anteriormente, se extrajo una sub-muestra estratégica de 25 imágenes representativas.

La selección de estas 25 imágenes no fue aleatoria, sino cualitativa, buscando cubrir la mayor varianza posible de la escena:

1. Frames con la pista despejada (para asentar la geometría del court).
2. Frames con aglomeración de jugadores en el área (para mejorar la separación de instancias).
3. Frames con el balón en movimiento rápido.

El objetivo es demostrar que, alimentando al sistema con solo el 3,4% de la información total (25 de 737 frames), el modelo es capaz de inferir correctamente el 96,6% restante del vídeo.

Con el "Dataset Base" (525 imágenes históricas) por un lado y el "Dataset MVP" (25 imágenes nuevas) por otro, surge el desafío técnico de fusionarlos sin corromper la estructura de datos. Para automatizar este proceso ETL (Extract, Transform, Load), se desarrolló el script code_1.py.

Este módulo es crítico para garantizar la integridad del entrenamiento. Su funcionamiento se divide en tres fases lógicas:

1. **Ingesta y Extracción:** El script localiza los archivos comprimidos (.zip) exportados desde Roboflow en Google Drive. Dado que Roboflow puede exportar versiones con nombres de carpetas distintos cada vez, el código normaliza la entrada extrayendo todo a un directorio temporal de trabajo.

2. **Validación de Clases y Metadatos (data.yaml):** El mayor riesgo al unir dos datasets distintos es la colisión de IDs. Por ejemplo, un dataset podría tener la clase ball en el índice 0, mientras que el otro la tiene en el 1. Si se mezclan directamente, el modelo aprendería ruido. code_1.py lee los archivos de configuración data.yaml de cada fuente y reescribe las rutas para que apunten a direcciones absolutas dentro del entorno de Google Colab (/content/...), asegurando que el entrenador YOLO encuentre las imágenes independientemente de dónde se ejecute el script.
3. **Fusión y Reestructuración de Directorios:** El script itera sobre las particiones train, valid y test, moviendo físicamente las imágenes y las etiquetas (.txt) a una estructura de Dataset Unificado. Al finalizar, genera un reporte de integridad que confirma que el número de imágenes coincide con el número de etiquetas, evitando errores de "archivo no encontrado" que detendrían el entrenamiento posterior.

Una vez ejecutado code_1.py, se dispone de un dataset consolidado de **1.259 imágenes** (tras aplicar Data Augmentation). Este conjunto unificado es el que alimenta al script de entrenamiento (Modelo1.ipynb). Al comparar los datasets antes de la fusión, se asegura que ambos comparten la misma topología de clases (los mismos 12 elementos). El re-entrenamiento con este dataset híbrido permite que el modelo mantenga su conocimiento general sobre el fútbol sala (adquirido en las 500 primeras fotos) mientras se "especializa" en las particularidades del nuevo vídeo gracias a las 25 imágenes inyectadas.

4.2. Motor de Inferencia (La Visión)

Una vez finalizada la fase de entrenamiento y obtenidos los pesos óptimos del modelo (**best.pt**), el sistema pasa a la etapa de explotación. Este módulo es el encargado de ejecutar el procesamiento visual del flujo de vídeo, actuando como el "ojo" del sistema. Desde una perspectiva arquitectónica, este componente abstrae la complejidad de la visión por computador, transformando una entrada de vídeo no estructurada (píxeles) en datos estructurados (clases y coordenadas geométricas), delegando la interpretación táctica a los módulos subsiguientes.

Para la ejecución, el script carga el modelo en la memoria de la GPU lo que permite realizar los cálculos matriciales necesarios para la inferencia en tiempos cercanos al tiempo real. Se ha configurado un umbral de **confianza (confidence threshold) del 0.25**, lo que significa que el motor descarta cualquier detección cuya probabilidad de certeza sea inferior al 25%, filtrando así el ruido visual y reduciendo los falsos positivos.

4.2.1. Ejecución sobre el Caso de Uso (MVP)

Para validar el sistema en un entorno de producción, se procesa el vídeo de prueba definido en el apartado anterior. Este clip, de 28 segundos de duración, es descompuesto por el motor en una secuencia discreta de **737 fotogramas** (frames).

El algoritmo de inferencia itera sobre esta secuencia imagen por imagen. En cada paso (Forward Pass), la red neuronal extrae las características visuales y propone múltiples detecciones. Posteriormente, se aplica un algoritmo de Supresión de No Máximos (NMS) para eliminar detecciones redundantes sobre un mismo objeto, quedándose únicamente con la predicción de mayor probabilidad.

El resultado es que, para cada uno de los 737 instantes del partido, el sistema es capaz de aislar la silueta exacta de los jugadores, la posición del balón y, lo más crítico, la geometría poligonal de la pista de juego, la cual varía en cada frame debido al movimiento de la cámara.

4.2.2. Serialización de Datos y Metadatos (.txt)

La salida principal de este módulo no es visual, sino numérica. El sistema serializa los resultados de la inferencia generando un archivo de texto (.txt) independiente para cada fotograma procesado.

Estos archivos actúan como una capa de persistencia intermedia, desacoplando la visión de la lógica de negocio. Cada línea del archivo representa un objeto detectado y sigue la estructura estándar de segmentación normalizada: [ID_CLASE] [x1] [y1] [x2] [y2] ... [xn] [yn]

- **ID_CLASE**: Entero que identifica la naturaleza del objeto (ej. 0 para el balón, 2 para el campo).
- **Coordenadas:** Lista de vértices normalizados (0-1) que definen el polígono exacto del objeto.

Gracias a este enfoque, se obtiene un registro histórico completo y estructurado de los 737 frames, permitiendo que los algoritmos de análisis táctico puedan trabajar sobre estos datos "crudos" sin necesidad de volver a procesar el vídeo.

4.2.3. Visualización y Verificación de Resultados

Paralelamente a la generación de datos, el motor produce una salida visual para la validación humana. El sistema genera una copia de cada frame sobre la que renderiza las máscaras de segmentación (overlays de colores), permitiendo verificar visualmente la precisión del modelo.

Adicionalmente, se realiza una proyección preliminar sobre el tablero táctico 2D para así ser utilizado en los siguientes pasos. Tanto los frames anotados como las proyecciones del tablero se almacenan automáticamente en una carpeta de resultados organizada. Esto permite al usuario inspeccionar la secuencia completa y validar que el modelo ha mantenido la consistencia temporal a lo largo de los 28 segundos del clip.

4.3. Analizador Semántico y Reconstrucción Geométrica

Tras la etapa de inferencia, el sistema dispone de una serie de detecciones geométricas precisas (polígonos de la pista, áreas, balón), pero carentes de contexto espacial. El algoritmo sabe dónde están los objetos en la imagen, pero desconoce en qué parte del terreno de juego se encuentran debido al movimiento constante de la cámara.

Este módulo, implementado en la clase *DetectorMejorado*, es el encargado de dotar de lógica al sistema, transformando detecciones aisladas en un escenario táctico

coherente mediante dos procesos fundamentales: la segmentación dinámica basada en referencias visuales y la persistencia temporal.

4.3.1. División Dinámica y Asignación de Lados

El desafío principal en una retransmisión de fútbol sala es la pérdida de referencia absoluta cuando la cámara hace zoom o paneos. Para solucionar esto, el algoritmo no confía en coordenadas fijas, sino que reconstruye la geometría del campo en cada fotograma.

El proceso se articula en torno a la creación de una "Línea Roja Virtual". Cuando el modelo detecta el medio campo (midfield), el código extrae sus vértices extremos (superior e inferior) y traza una recta vectorial que divide la pantalla en dos hemisferios. Esta línea actúa como una frontera matemática dinámica que permite realizar dos operaciones críticas:

1. **Corte del Polígono de Pista:** El sistema toma la detección general de la pista (court) y la secciona geométricamente utilizando la ecuación de la recta. Los vértices situados a la izquierda conforman el nuevo polígono court_1 (campo local) y los de la derecha el court_2 (campo visitante).
2. **Contextualización de Elementos Estáticos:** Esta lógica de división se extiende a todos los elementos de infraestructura. El algoritmo calcula el centroide de cada area, goal (portería) y corner detectado. Comparando su posición respecto a la Línea Roja, el sistema los renombra automáticamente:
    - Si una portería cae en el lado negativo de la recta, se etiqueta como goal_1.
    - Si un área cae en el lado positivo, se transforma en area_2.

Gracias a esto, pasamos de tener etiquetas genéricas a tener entidades semánticas posicionadas (area_1 vs area_2), lo cual es indispensable para que el tablero táctico posterior sepa dónde dibujar cada línea.

4.3.2. Persistencia Temporal y Generación de Datos Semánticos

La segunda problemática abordada es la estabilidad. En situaciones de juego real, es común que la línea de medio campo desaparezca del encuadre (por un zoom a un córner) o que el modelo de detección parpadee debido a oclusiones (jugadores tapando las líneas).

Para mitigar esto, el *DetectorMejorado* implementa un motor de memoria y consenso. El sistema mantiene un historial de los últimos 5 fotogramas procesados. Si en el frame actual se pierde la referencia visual (la Línea Roja), el algoritmo no deja el mapa en blanco; en su lugar, consulta el estado anterior y "recuerda" en qué sector del campo se encontraba la cámara. Si la detección es inestable, aplica un consenso con los frames previos para suavizar la geometría y evitar saltos visuales.

Finalmente, este módulo consolida toda la información generando una nueva estructura de datos. Los archivos de salida (.txt) ya no contienen las clases originales de YOLO, sino las clases semánticas procesadas. Un ejemplo de la salida real de este módulo sería:

court_2 0.923 0.295 ... (Polígono del lado derecho)
area_2 0.948 1.000 ... (Área visitante identificada)
goal_2 0.980 0.450 ... (Portería visitante)
player-team1 0.824 ... (Jugador posicionado en este contexto)

Este paso cierra la brecha entre la visión pura (píxeles) y la táctica (posiciones), entregando al siguiente módulo una representación estructurada y completa del estado del partido.

4.4. Proyector Geométrico (Las Matemáticas)

Una vez que disponemos de los datos semánticos estructurados (sabemos qué es cada objeto y en qué lado del campo está), llegamos al núcleo matemático del TFG. El objetivo de este módulo es realizar la transformación homográfica inversa: traducir la posición de un píxel en la pantalla (u, v) a una coordenada métrica real en el suelo (x, y).

Sin embargo, para evitar que este sistema sea una "caja negra" —un algoritmo que escupe números sin garantía de precisión—, se estableció un requisito de validación visual: la Telemetría en Tablero Digital. Dado que en el fútbol sala es estándar el uso de tableros tácticos 2D para el análisis, el sistema debe ser capaz de replicar lo que ocurre en el vídeo sobre este plano virtual. Si la representación en el tablero coincide con la realidad del vídeo, las matemáticas son correctas.

Para lograr esto, el proceso se divide en tres fases secuenciales: determinar la ubicación física de la cámara (estática), calcular su ángulo de visión frame a frame (dinámica) y, por último, transformar a los jugadores en el tablero.

4.4.1. Calibración Estática: Localización de la Cámara

Un error común en los primeros prototipos fue asumir que la cámara siempre estaba centrada y en una posición elevada estándar. La realidad de los vídeos amateurs es mucho más caótica: grabaciones desde esquinas, a pie de pista, o desplazadas hacia un fondo. Si el sistema asume una posición errónea, la proyección sale deformada.

Para solucionar esto sin intervención manual, se desarrolló el algoritmo *AnalizadorCompleto.* Este módulo no analiza un frame aislado, sino que realiza una triangulación estadística sobre la totalidad del vídeo (los 737 frames en este caso de estudio).

A. Análisis de Orientación (Horizontal vs. Vertical)

El primer paso es saber si la cámara está grabando desde un fondo (detrás de la portería) o desde la banda (lateral). El sistema no "ve" la cámara, pero puede deducir su posición basándose en qué partes del campo son visibles a lo largo del tiempo.

El algoritmo construye un histograma de frecuencias de las clases estructurales (court_1, court_2, midfield).

1. **Conteo de Apariciones:** Se analiza cada uno de los 737 frames. Si en un frame el modelo detecta el campo izquierdo, suma un punto a court_1; si ve el centro, suma a midfield, etc.
2. **Umbral de Significancia:** Se establece un filtro del 10% del total de frames para descartar detecciones erróneas o esporádicas.
3. **Árbol de Decisión Lógico:**
    - Caso Horizontal (Fondo): Si el reporte indica que court_1, court_2 y midfield tienen una alta frecuencia de aparición, significa que la cámara ha sido capaz de barrer todo el campo de izquierda a derecha. Físicamente, esto solo es posible si la cámara está ubicada en uno de los Fondos, perpendicular a la línea de banda.
    - Caso Vertical (Banda): Si el reporte muestra una predominancia masiva de un solo lado (ej. 90% court_1) y casi nula del otro, el sistema deduce que la cámara está en la Banda. Al estar en un lateral, la perspectiva cónica impide ver el lado lejano del campo con claridad.

En nuestro caso de uso (MVP), el sistema detectó ambos lados y el medio con alta frecuencia, clasificando correctamente la orientación como "Horizontal".

B. Triangulación de la Posición (X, Y) mediante "Centros de Masa"

Una vez determinada la orientación general (Fondo vs. Banda), el sistema se enfrenta al reto de calcular la coordenada métrica exacta del trípode. ¿La cámara está alineada perfectamente con el centro de la portería o está desplazada unos metros hacia un lateral? Responder a esto es crucial para evitar deformaciones en la proyección del tablero.

Para resolverlo sin intervención manual, se ha implementado un algoritmo basado en el Centro de Masa Visual. La premisa es sencilla: el operador de cámara, de manera instintiva, tiende a mantener la acción principal del juego dentro del encuadre. Dado que estadísticamente la acción oscila alrededor del círculo central, el promedio de todas las detecciones nos indica hacia dónde ha estado apuntando la lente la mayor parte del tiempo.

El cálculo se realiza en cuatro pasos, siendo el tercero el más crítico para entender la geometría del sistema:

1. Cálculo del Centroide por Frame: En cada uno de los 737 fotogramas, el sistema calcula el punto medio (x, y) de todas las detecciones (jugadores, balón, árbitro).

2. Promedio Global (μ): Se obtiene la media de todos estos puntos. Imaginemos un vídeo estándar de 1280 píxeles de ancho, donde el centro matemático es el píxel 640. El algoritmo verifica dónde cae el promedio real de la acción (por ejemplo, en el píxel 800).

3. Aplicación de la Lógica Inversa (El Principio de Compensación): Aquí es donde el algoritmo deduce la posición física basándose en la óptica. Para entenderlo, usemos un ejemplo práctico:

Tras esto, vamos a ver un ejemplo para que sea más fácil de entender. Imaginemos que un operador de cámara está grabando una obra de teatro y quiere enfocar al protagonista, que está en el centro exacto del escenario.

- **Caso Central:** Si el operador está sentado en la butaca central, solo tiene que apuntar recto. El protagonista aparecerá en el centro de la imagen.
- **Caso Desplazado (Lógica Inversa):** Si el operador está sentado en el extremo izquierdo del teatro, para poder grabar al protagonista (que sigue en el centro), está obligado a girar su cámara hacia la derecha.

El algoritmo detecta precisamente este giro. Si observa que el "grueso" de la acción aparece sistemáticamente desplazado hacia la derecha en la imagen de vídeo (píxeles altos), deduce geométricamente que la cámara física debe estar situada a la izquierda del eje central para necesitar ese ángulo de visión.

Por tanto, la regla aplicada es inversa:

- Desplazamiento visual a la ➡ Derecha Corrección de posición a la Izquierda.
- Desplazamiento visual a la ➡ Izquierda Corrección de posición a la Derecha.

4. Fórmula de Corrección y Resultado: El sistema traduce esta deducción lógica a una coordenada en el tablero mediante la fórmula:

$$Pos_{tablero} = Centro_{tablero} - (Promedio_{video} - Centro_{video}) \times K$$

Finalmente, se calcula la desviación estándar (σ) para validar la estabilidad de la grabación. En el caso de uso de este MVP, el algoritmo detectó que el centro de acción estaba ligeramente desplazado a la derecha en el vídeo. Aplicando la lógica inversa, corrigió la posición de la cámara en el tablero desde el centro teórico (X=300) a una posición real de X=283 (ligeramente a la izquierda), validando así la precisión del sistema para auto calibrarse.

De todas formas, hay que tener en cuenta que esta solución se ha validado con un caso de uso concreto y habría que probarla en más contextos para confirmar que es infalible. Por ejemplo, en un plano general donde se vea el campo entero, siempre va a detectarlo como que ha sido grabado desde el fondo, pero puede haber casos extremos donde desde una grada muy alta y una lente especial, se vea también el campo entero desde la banda. Para solucionar esto de forma sencilla, nos podemos apoyar en la 'Línea Roja' del medio campo que calculamos antes: su inclinación es el mejor chivato. Si vemos que esa línea aparece vertical en la imagen, seguro que estamos en la banda; si es más bien horizontal, estamos en el fondo. Por otro lado, en casos extremos como cámaras fijas en una esquina con gran angular, la lógica debería seguir funcionando por su propia naturaleza: aunque la cámara no se mueva, el grueso de la acción no aparecerá en el centro de la imagen, sino desplazado, y eso hará que la fórmula matemática detecte el desvío y coloque la cámara virtual en la esquina correcta del tablero automáticamente.

4.4.2. Proyección Dinámica: El Cono de Visión (FOV)

Llegamos a una de las fases más complejas y determinantes de todo el proyecto. Una vez estimada la posición física de la cámara (el trípode), el sistema debe resolver la dinámica del movimiento: saber exactamente hacia dónde está apuntando la lente en cada milésima de segundo. Este paso ha supuesto el mayor desafío técnico del TFG, obligando a replantear las hipótesis iniciales y a simplificar los cálculos matemáticos para evitar que el sistema fallara por sobre-complejidad.

El principal obstáculo encontrado fue un malentendido conceptual sobre el comportamiento óptico de la cámara. En los primeros prototipos, se diseñó un algoritmo que calculaba el Campo de Visión (FOV) o "Zoom" de manera dinámica, asumiendo que este cambiaba frame a frame. El resultado fue desastroso: el cono de visión se ensanchaba y estrechaba erráticamente, haciendo imposible una proyección coherente.

Tras un análisis exhaustivo de las grabaciones, se descubrió el origen del error: el operador de cámara no hace zoom. En el fútbol sala amateur, es habitual fijar una distancia focal y mantenerla para no perder el foco con el movimiento rápido. Sin embargo, se produce una ilusión óptica muy potente que confunde al algoritmo:

- Cuando la cámara enfoca al centro del campo (perpendicular a la banda), la pista llena casi toda la imagen, dando la sensación de un ángulo muy abierto.
- Cuando la cámara gira hacia las porterías (diagonal), la perspectiva hace que la pista ocupe menos píxeles y aparezca más "fondo" (paredes, gradas), creando la falsa impresión de que el ángulo se ha cerrado o que hay más zoom.

Comprender que el ángulo de la lente es, en realidad, constante (estático) y que lo único que cambia es la rotación, fue la clave para estabilizar el sistema.

A. La Solución de Fuerza Bruta: Optimización del Espacio de Parámetros

Dado que carecemos de los metadatos intrínsecos de la cámara (distancia focal, distorsión de lente, ubicación exacta), el sistema debe inferirlos mediante simulación. Para ello, la clase *CalibradorOptimo* ejecuta un algoritmo de búsqueda exhaustiva (Grid Search) que prueba múltiples combinaciones posibles hasta encontrar la que mejor encaja con la realidad.

El objetivo es minimizar una **Función de Coste (J)** que mide la discrepancia entre la proyección teórica y la realidad observada.

1.Definición del Espacio de Búsqueda: El algoritmo define una malla (grid) de hipótesis plausibles para un partido de fútbol sala estándar:

- <u>Posiciones (P):</u> Se definen tres anclajes estratégicos en el eje X:
    - *0*: La Esquina Izquierda

- *300*: En el Centro exacto
- *600*; La Esquina Derecha

- Campo de Visión (FOV): Se evalúa un rango de aperturas angulares típicas:
  - Desde 30º, lo cual es una lente cerrada
  - Hasta 60º, lo cual es un gran angular.

2.Justificación de la Simplificación (X=300): Aunque en el apartado 4.4.1 el análisis estadístico arrojó una posición precisa de X=283, este apartado de crear el cono de visión y los siguientes pasos conllevan una gran complejidad matemática y se han producidos muchos errores a lo largo del desarrollo, por lo que se decidió simplificar los cálculos dos únicamente estas opciones. Aun así, el siguiente paso sería el de incluir la lógica anterior a estos cálculos para obtener un sistema mejorado y más realista.

3. Selección del Modelo Ganador (¿Por qué 30º?): El sistema itera sobre 100 fotogramas aleatorios. Para cada configuración de FOV, proyecta los elementos del vídeo al tablero y mide la distancia entre donde caen y donde deberían estar.

$$J(P, FOV) = \sum_{k=1}^{N} \left| \text{Pos}_{real}^{(k)} - \text{Proyección}(P, FOV, \text{Detección}^{(k)}) \right|$$

Tras el análisis, el algoritmo determinó automáticamente que el FOV óptimo es de 30º. ¿Por qué eligió este valor y qué implicaciones físicas tiene?

Un FOV de 30º corresponde a una lente cerrada o Teleobjetivo (Zoom), en contraposición a los 60º o más de un Gran Angular (tipo GoPro). La elección del algoritmo se basa en cómo se proyectan los objetos situados en los bordes de la imagen:

- **La Hipótesis del Gran Angular (60º):** Si el algoritmo intentaba proyectar usando 60º, asumía que la cámara abría mucho el campo. Matemáticamente, esto provocaba que un objeto situado en el borde de la pantalla se interpretara como si estuviera muy lejos lateralmente. Al llevarlo al mapa, ese objeto "se salía" del estadio o aparecía metros más allá de la línea de banda. El error acumulado era enorme.

- **La Hipótesis del Zoom (30º):** Al probar con 30º, el algoritmo asumió que la cámara estaba haciendo "zoom" desde lejos. En esta configuración, los objetos del borde de la pantalla no se proyectan tan lejos lateralmente, sino que se mantienen más agrupados. Al hacer esto, las líneas proyectadas coincidieron casi perfectamente con las líneas reales del mapa JSON.

B. Cálculo Trigonométrico de la Rotación (θ)

Con los parámetros fijos (FOV=30º, X=300), el sistema reduce el problema a una única incógnita dinámica: ¿Hacia dónde está girada la cámara en este instante exacto?
Para resolverlo, utilizamos Trigonometría Inversa Proyectiva. La técnica consiste en identificar "Puntos de Anclaje" (A) cuya posición real conocemos (ej. el centro del medio campo) y medir cuánto se han desplazado en la pantalla.

1. La Fórmula de Proyección Angular: La lógica es la siguiente: sabemos dónde está un objeto en el tablero (su ángulo real). Si en la cámara no aparece centrado, la diferencia entre "dónde debería estar" y "dónde lo veo" es exactamente lo que ha girado la cámara.
Sea:

- x_{norm}: La posición del objeto en la imagen (de 0.0 a 1.0).
- $offset$: El desplazamiento desde el centro de la imagen ($x_{norm} - 0.5$).
- θ_{obj}: El ángulo real del objeto en el tablero respecto a la cámara.

Primero, calculamos el ángulo de corrección visual (α). Esta fórmula convierte la distancia en píxeles (offset) a grados de rotación, basándose en la apertura de la lente (FOV):

$$\alpha = \arctan\left(2 \cdot offset \cdot \tan\left(\frac{FOV_{rad}}{2}\right)\right)$$

Explicado más sencillamente, si el objeto está desplazado un 25% hacia el borde de la foto y mi lente abre 30 grados, eso significa que el objeto está visualmente a

7.5 grados del centro. Finalmente, obtenemos la orientación de la cámara restando esta corrección al ángulo real:

$$\theta_{cam} = \theta_{obj} - \alpha$$

<u>Ejemplo Numérico Paso a Paso:</u>

1. *Situación*: La cámara está en el centro del tablero (X=300). El sistema detecta el Medio Campo.
2. *La Realidad* θ_{obj} : Sabemos por el mapa que el medio campo está justo enfrente de la cámara. Su ángulo es 90º (recto hacia adelante).
3. *La Visión (offset):* En el vídeo, el medio campo no aparece en el centro, sino desplazado a la izquierda (en el píxel 320 de 1280, offset de -0.25).
4. *El Cálculo (α):* El sistema pregunta: "¿Cuántos grados son ese desplazamiento de píxeles?". Con un zoom de 30º, la fórmula nos dice que son -7.5º.
5. *El Resultado* θ_{cam}

$$\theta_{cam} = -90^\circ - (-7.5^\circ) = -82.5^\circ$$

6. *Interpretación:* Si el objeto que debería estar en el centro aparece a mi izquierda, es porque yo (la cámara) he girado la cabeza hacia la derecha. El sistema acaba de calcular ese giro exacto.

2. Fusión de Sensores Visuales (Media Ponderada): A menudo, el sistema ve varios puntos a la vez (ej. el medio campo y el área). ¿De cuál nos fiamos más? Las lentes de las cámaras suelen deformar la imagen en los bordes (efecto barril), por lo que los objetos en el centro son más "fiables". Para obtener una rotación robusta, calculamos una Media Ponderada:

1. *Asignamos un peso w_i a cada objeto*. Si el objeto está en el centro de la pantalla, w=1 (confianza máxima). Si está en el borde, el peso baja casi a 0.

$$w_i = 1 - |x_{norm}^{(i)} - 0.5|$$

2. *Sumamos los vectores* de todos los objetos detectados ponderados por su confianza.
3. *El resultado* es un ángulo de consenso que prioriza siempre lo que la cámara tiene enfrente, ignorando las deformaciones de los extremos.

C. Validación Visual: El Cono de Telemetría

Para verificar que las matemáticas reflejan la realidad, se genera una representación gráfica sobre el tablero táctico: el Cono de Visión. Se dibuja un triángulo de luz translúcido (amarillo) que nace en la posición de la cámara y se abre 30° (el FOV calculado).

El resultado final es un vídeo de telemetría donde este cono actúa como un "gemelo digital" de la cámara real:

- Si el operador sigue un contraataque hacia la derecha, el cono en el mapa barre hacia la derecha iluminando la zona de juego correspondiente.
- Si la cámara vuelve al centro, el cono regresa sincronizado.

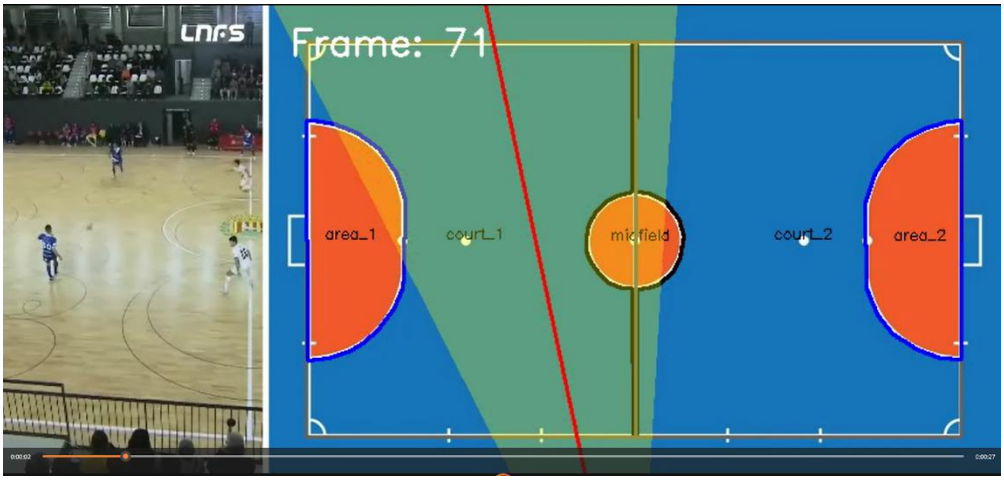

Ilustración 12. Cono de Telemetría

Esta sincronización visual es la prueba definitiva de que hemos superado el problema de la perspectiva. Confirma que el sistema ha "entendido" la geometría de la escena, permitiendo pasar a la fase final: ubicar a los jugadores dentro de ese cono con coordenadas métricas reales. De todas formas, hay que tener en cuenta que esto es un MVP y, aunque funciona, tiene margen de mejora. De cara a futuro, el objetivo sería conectar los pasos de forma directa: en vez de volver a estimar dónde está la cámara en este punto, deberíamos coger el dato exacto que ya calculamos en el apartado anterior (ese X=283) y usarlo aquí directamente. Además, con el tema del zoom (FOV), ahora mismo el sistema prueba unos cuantos valores estándar para no tardar demasiado. Si quisiéramos una precisión milimétrica, tendríamos que 'afinar' mucho más esa búsqueda, probando un rango de ángulos mucho más amplio y con saltos más pequeños para dar con el valor exacto de la lente y tener en cuenta el problema que hay con los ángulos distorsionados de los laterales.

4.4.3. Mapeo de Entidades: Algoritmos de Proyección y Desafíos Técnicos

Llegamos a la etapa final de la cadena de procesamiento: la transposición de las coordenadas (u, v) de la imagen a las coordenadas métricas (X, Y) del tablero. A primera vista, podría parecer un problema trivial de geometría, pero la realidad de las grabaciones deportivas no controladas (cámara en mano, zoom variable, perspectivas laterales) convierte este paso en el mayor cuello de botella del sistema. Para entender la solución final, es fundamental analizar primero por qué fallaron las aproximaciones matemáticas tradicionales.

A. Antecedentes: La Trampa de la Complejidad Matemática

En el desarrollo de sistemas de visión artificial, la intuición inicial siempre apunta hacia soluciones de álgebra lineal compleja. Durante la fase de investigación de este TFG, se intentaron implementar dos métodos "de libro" que resultaron inviables en la práctica:

1. **El Fracaso de la Homografía Planar:** La primera opción lógica fue calcular una matriz de Homografía que relacione los puntos del plano imagen con el plano del tablero. Para ello, el algoritmo intentaba buscar correspondencias de puntos clave

(keypoints): "la intersección de la línea de medio campo con la banda", "la esquina del área", etc.

- El Problema: En el fútbol sala, debido al tamaño de la pista y la cercanía de la cámara, es extremadamente raro ver 4 puntos de referencia al mismo tiempo. A menudo, el modelo solo ve "un trozo de línea curva" (el círculo central) o "una línea recta blanca" sin saber si es la banda o el área. Al no poder identificar puntos únicos y estables, la matriz H no podía resolverse o generaba proyecciones incorrectas donde los jugadores aparecían fuera del estadio.

2. **El Intento de la Malla Global (Stitching)**: Se intentó reconstruir una panorámica completa del campo ("malla") uniendo todos los frames del vídeo para luego proyectar sobre ella.
   - El Problema: Fue un "deseo imposible". El movimiento constante de la cámara, sumado al desenfoque de movimiento (motion blur) y los cambios de iluminación, impedían crear una malla coherente. Las imágenes no "cosían" bien, y la transformación matricial resultante acumulaba errores de deriva (drift) inaceptables tras pocos segundos de vídeo.

B. La Solución Final: Proyección Relativa Híbrida

Tras descartar las soluciones globales por falta de estabilidad, se optó por una estrategia local mucho más inteligente: resolver la posición de cada jugador basándose en su contexto inmediato. En lugar de intentar mapear todo el estadio a la vez, el sistema se pregunta: "¿En qué porcentaje del campo está este jugador?". Esta técnica combina dos métodos que actúan en jerarquía: el Escáner Vertical (para el campo abierto) y los Anclajes de Precisión (para las áreas).

1. El Algoritmo del "Escáner Vertical" (La Regla del Porcentaje): Este método se basa en una premisa geométrica simple: aunque la perspectiva deforme el tamaño de los objetos, la relación de profundidad vertical se mantiene constante. Para entenderlo, imaginemos que la imagen es una escalera.

- El sistema analiza la columna de píxeles donde están los pies de un jugador x_{img}, y_{img}

- Mira la máscara de segmentación (lo que la IA detectó como suelo) para encontrar el primer píxel de pista (y_{top}) y el último (y_{bottom}).

Ejemplo Práctico:

Imaginemos una columna de la imagen donde la pista empieza en el píxel 200 (línea de banda superior) y termina en el píxel 800 (línea de banda inferior). La altura total de la pista visual es de 600 píxeles. Si detectamos a un jugador con los pies en el píxel 500, el sistema hace el siguiente razonamiento:

> *"El jugador está 300 píxeles por debajo del inicio. Eso es exactamente el 50% del recorrido (300 de 600). Por tanto, en el tablero táctico, debo colocarlo en el centro exacto (50%) de la anchura del campo."*

Esta lógica se formaliza con la siguiente ecuación de normalización:

$$Ratio_{profundidad} = \frac{y_{img} - y_{top}}{y_{bottom} - y_{top}}$$

$$Y_{mapa} = Y_{tablero_top} + (Ratio_{profundidad} \times Altura_{tablero})$$

2. Refinamiento Híbrido por Anclajes (El "GPS Local"): El método anterior funciona genial cuando se ve todo el campo, pero... ¿qué pasa si la cámara hace zoom y corta la línea de banda inferior?

En ese caso, el valor (y_{bottom}) sería incorrecto (sería el borde de la pantalla, no la línea real), y nuestro cálculo del "50%" fallaría estrepitosamente, enviando al jugador fuera del estadio. Para solucionar esto en la versión final (CLEAN_OUTPUT), se implementó un sistema de Anclajes (Anchors) que actúa como una jerarquía de decisión inteligente:

- *Paso 1:* ¿Está en zona segura (Área)? Antes de medir nada, el algoritmo comprueba si el jugador se encuentra visualmente dentro de un polígono de área (area_1 o area_2). Las áreas son "zonas seguras" porque conocemos sus medidas exactas gracias al archivo estructura_dual.json. Si el jugador está en el área, el sistema ignora la máscara visual (que puede

tener errores) y utiliza los límites matemáticos del área como y_{top} y y_{bottom}. Es como usar un "GPS local" de alta precisión: da igual si la cámara corta el campo, si estás en el borde del área pequeña, el sistema sabe exactamente a qué metro corresponde eso.

- *Paso 2*: Fallback (Campo Abierto) Solo si el jugador está en "tierra de nadie" (zona central del medio campo sin líneas cercanas), el sistema recurre al método del escáner vertical explicado antes.

Esta arquitectura híbrida es la clave del éxito del proyector: garantiza precisión milimétrica en las zonas críticas (las áreas, donde ocurren los goles) y mantiene una estimación robusta en el resto del campo, evitando que los jugadores "salten" cuando la cámara se mueve rápido.

C. Casos Especiales: El Balón y el Suavizado de Trayectorias

Existen dos elementos adicionales que requirieron un tratamiento específico para que la visualización final fuera útil para un analista:

1. La Interpolación del Balón (The Ball Problem): El balón es el objeto más difícil de procesar: es pequeño, se mueve rápido (generando blur) y a menudo queda oculto tras los jugadores. Esto provoca que la detección sea intermitente (aparece en el frame 10, desaparece hasta el 15, vuelve en el 16).
Para solucionar el "parpadeo", el sistema implementa una Interpolación Lineal Temporal en la función precalcular_trayectoria_bola: si el balón se detecta en el instante A y en B, el sistema rellena matemáticamente las posiciones intermedias asumiendo una velocidad constante.

Cabe mencionar que esta solución proyecta el balón asumiendo que está "pegado al suelo" (proyección de su sombra). Cuando el balón vuela por el aire, la perspectiva 2D hace que parezca estar más "atrás" en el campo. Resolver la altura (Eje Z) con una sola cámara requeriría modelos físicos de gravedad complejos que escapan al alcance de este MVP, por lo que se asume la proyección en planta como estándar aceptable.

2.Suavizado de Movimiento (Smoothing): Las detecciones crudas de redes neuronales tienen un ligero "temblor" natural (ruido de alta frecuencia) que hace que los puntos en el tablero vibren. Para ofrecer una visualización profesional, se aplica un filtro de suavizado exponencial (Exponential Moving Average) a las coordenadas finales:

$$Pos_{final} = (\alpha \cdot Pos_{nueva}) + ((1 - \alpha) \cdot Pos_{anterior})$$

Con un factor α=0.6, logramos que los jugadores se deslicen por el tablero de manera fluida, eliminando el ruido digital sin introducir un retardo (lag) perceptible en la posición.

4.5. Sintetizador Táctico: Consolidación y Generación de Entregables

Como cierre de cada etapa de procesamiento descrita anteriormente, el sistema genera automáticamente un archivo de vídeo renderizado. Esta decisión de diseño es fundamental, ya que permite validar visualmente conjuntos masivos de datos numéricos: al ver los 737 frames reproducidos en secuencia sobre el tablero, cualquier error matemático se manifiesta como un "salto" o inconsistencia visual, permitiendo una depuración inmediata y global del partido. Además, en este video se han ido incluyendo distintos debugs para comprobar que todos los pasos funcionan correctamente. Esto se puede ver en un ejemplo como:

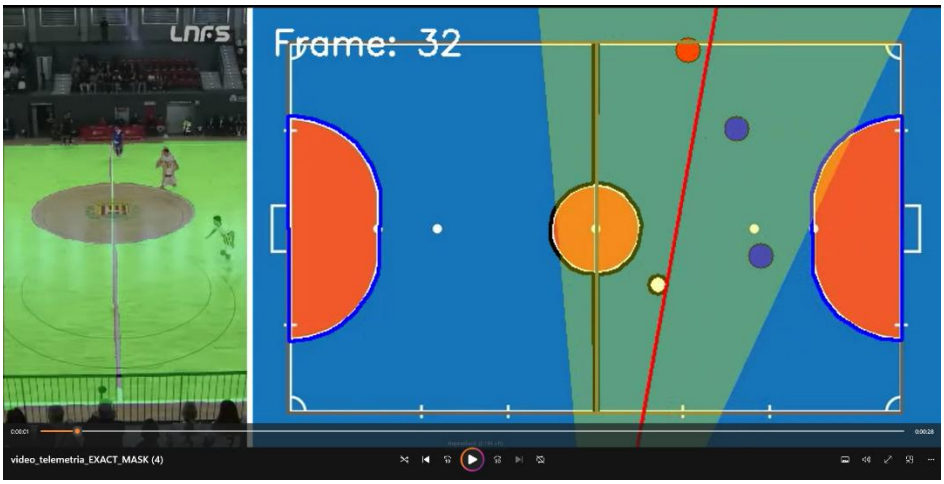

Ilustración 13. Vídeo de Salida Con Debugs

Aun así, como resolución final se han quitado todos los debugs para simular como se vería finalmente. Esto se ve:

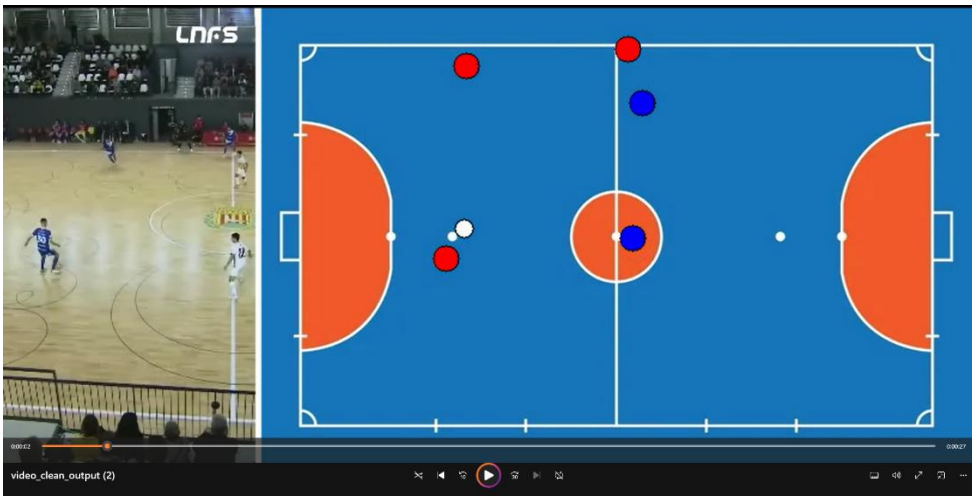

Ilustración 14. Vídeo de Salida Sin Debugs

Hasta el módulo anterior, el sistema cumplía con el objetivo estricto del **Producto Mínimo Viable (MVP)**: proyectar en el tablero todo aquello que la cámara estaba viendo en ese instante. Sin embargo, para que esta herramienta tenga una utilidad real en el análisis deportivo, es necesario dar un paso más allá. Un entrenador no necesita saber únicamente dónde están los cuatro jugadores que encuadra la cámara; necesita conocer la disposición táctica de los **10 jugadores simultáneamente**, independientemente de si el operador de cámara ha hecho zoom sobre el balón o está grabando solo una banda.

El objetivo final de este módulo es, por tanto, reconstruir la **omnisciencia** del partido. Esto implica mantener la posición de los jugadores que salen de plano (persistencia) y agrupar las detecciones fragmentadas para que el sistema entienda que un jugador que sale por la derecha y reaparece es la misma entidad, y no uno nuevo.

4.5.1. La Estrategia del "Frame 0" y la Mirada al Futuro

Para lograr esta consistencia total, el algoritmo se ancla en el **Frame 0**. Este momento es crítico porque define la "alineación titular" sobre el tablero. El problema reside en que, si solo nos fiáramos del presente, empezaríamos el análisis con una formación incompleta de 4 o 5 jugadores.

Para solucionar esto, el sistema implementa una lógica de **Proyección Temporal Inversa**. En lugar de mirar solo el fotograma actual, el algoritmo "mira al futuro cercano" (analiza los primeros segundos posteriores al inicio). La premisa es sencilla: en cuanto el balón ruede, la cámara se moverá o abrirá el plano para seguir el juego. El sistema detecta a los jugadores que aparecen en esos momentos futuros, calcula su posición y traza una trayectoria inversa hacia atrás en el tiempo para deducir dónde estaban parados exactamente en el Frame 0. De esta manera, logramos colocar a los 10 jugadores en el tablero inicial antes incluso de que la cámara los haya enfocado oficialmente.

4.5.2. Implementación Técnica: El Código AnalizadorTactico

Esta lógica deductiva se materializa en el script *AnalizadorTactico*. A diferencia de los códigos anteriores de visualización pura, este script no se limita a pintar puntos, sino que gestiona una lista de estado persistente (el Roster) y aplica capas de inferencia para rellenar los huecos de información.

A. Inicialización y Gestión del Roster

El código comienza definiendo la estructura ideal del partido. La función _inicializar_roster crea una plantilla vacía con los 10 espacios necesarios (5 locales y 5 visitantes). Esta lista actúa como el "inventario" que el algoritmo debe completar obligatoriamente, asegurando que no se generen ni más ni menos jugadores de los reglamentarios.

B. Motor de Inferencia por Capas

El núcleo del procesamiento reside en la función ejecutar_analisis, la cual aplica una arquitectura secuencial de cuatro capas para deducir la posición de los 10 jugadores. Este sistema no se limita a registrar lo que ve, sino que utiliza reglas lógicas y temporales para completar la información que falta:

- **Capa 1: La Realidad Observable**. En esta primera fase, el sistema asienta lo indiscutible proyectando las detecciones visuales del Frame 0. Es el inventario base: todo lo que la visión por computador detecta con alta confianza se coloca en

el tablero. Crucialmente, en este paso se fija la posición inicial del balón (self.ball_pos), estableciendo el punto de partida geográfico de la narrativa del partido. Si el modelo ve a dos jugadores en el círculo central sacando, estos se marcan como CONFIRMADO_REAL, convirtiéndose en los pilares inamovibles sobre los que se construirá el resto de la formación.

- **Capa 2: Lógica de Porteros.** Inmediatamente después, se aplica conocimiento del dominio deportivo. Dado que es muy habitual que la cámara se cierre sobre el círculo central durante el saque inicial, dejando las porterías fuera de plano, el sistema verifica si estos roles (goalkeeper-team1 y goalkeeper-team2) están vacíos en el inventario inicial. Si no se han detectado visualmente, el algoritmo los posiciona automáticamente en sus coordenadas canónicas bajo los tres palos. Estos jugadores se marcan con el estado INFERIDO_LOGICA, indicando que su presencia es una deducción basada en las reglas del juego y no una observación directa.
- **Capa 3: Barrido Futuro (Time-Travel).** Para localizar a los jugadores de campo restantes que están fuera de plano, el sistema ejecuta una proyección inversa iterando sobre los siguientes 150 fotogramas (aprox. 5 segundos). La premisa es que, al iniciar el juego, la cámara ampliará el plano o realizará un paneo, revelando zonas ocultas. Si el sistema detecta en ese futuro cercano a un jugador estable que no estaba presente al inicio, calcula su posición y lo "importa" hacia atrás en el tiempo al Frame 0. Por ejemplo, si un extremo aparece en la banda en el segundo 3, el sistema deduce que ya estaba allí parado en el segundo 0 y lo añade al tablero inicial con la etiqueta INFERIDO_FUTURO.
- **Capa 4: Auditoría de Conflictos (El "Fantasma").** Finalmente, se ejecuta una auditoría geométrica para mitigar el problema de los duplicados en los bordes del cono de visión. Cuando hay un ligero movimiento de cámara, un jugador situado en el límite puede entrar y salir del encuadre, provocando que el sistema lo confunda con otro cercano o lo duplique (creando un "fantasma"). Para solucionarlo, el algoritmo verifica si algún jugador "traído del futuro" cae geométricamente en una zona que ya estaba siendo grabada y aparecía vacía en el inicio. Si esto ocurre, el sistema deduce que se trata de un error de detección previo y fusiona los datos, realizando un INTERCAMBIO de identidades y priorizando la coherencia espacial para evitar que aparezcan más jugadores de los permitidos.

Aun así, se sabe que esto es un primer paso para un sistema más grande, pero que ya quedaría fuera de este trabajo. El siguiente paso sería escalar esta lógica a la totalidad del vídeo, lo cual presenta una dificultad mayor: el movimiento lateral de la cámara. Actualmente, cuando la cámara realiza un barrido de un área a la otra, los jugadores salen de plano y el sistema pierde su rastro (track ID). Al volver a enfocar la zona inicial, el sistema no puede garantizar si son los mismos jugadores o si han permutado posiciones. La solución propuesta para el futuro implica una lógica de Inferencia Continua, donde se utilizaría el "viaje al futuro" de manera recurrente para deducir la identidad de un jugador que reaparece basándose en su comportamiento posterior o asignándole la identidad del jugador que ocupaba esa zona anteriormente.

C. Renderizado Final (CLEAN_OUTPUT)

Por último, el resultado de todo este proceso es la generación de los dos entregables finales: el Vídeo de Telemetría completo y una Imagen Estática del planteamiento inicial con todos los jugadores posicionados (distinguiendo entre datos reales e inferidos mediante etiquetas como RECUP o NUEVO), la cual es:

```
--- POSICIONES FINALES ---
goalkeeper-team1 -> (80,190) [INFERIDO_LOGICA] Origin:
player-team1 -> (410,333) [CONFIRMADO_REAL] Origin:Frame 0
player-team1 -> (302,54) [INFERIDO_FUTURO] Origin:F28
player-team1 -> (136,82) [INFERIDO_FUTURO] Origin:F37
player-team1 -> (152,213) [INFERIDO_FUTURO] Origin:F41
goalkeeper-team2 -> (520,190) [INFERIDO_LOGICA] Origin:
player-team2 -> (450,272) [CONFIRMADO_REAL] Origin:Frame 0
player-team2 -> (359,186) [CONFIRMADO_REAL] Origin:Frame 0
player-team2 -> (451,110) [CONFIRMADO_REAL] Origin:Fix por F28
player-team2 -> (362,96) [INFERIDO_RECUPERADO] Origin:Recuperado F0
BALL START -> (407,296) Origin:Frame 0
```

Ilustración 15. Frame 0

5. Pruebas y Resultados

La exposición de las pruebas y resultados experimentales se ha estructurado siguiendo el mismo orden cronológico en el que fueron ejecutadas durante el desarrollo del proyecto. Esta decisión narrativa busca no solo presentar los datos finales, sino permitir la comprensión de la evolución técnica del sistema, justificando de manera objetiva las decisiones que se tomaron en cada iteración basándose en el rendimiento observado. A tal efecto, el presente capítulo aborda el análisis desde dos perspectivas complementarias: una primera evaluación **cuantitativa**, centrada en las métricas de rendimiento del modelo de Inteligencia Artificial en su entorno de entrenamiento, y una segunda evaluación **cualitativa y funcional**, orientada a verificar la utilidad real del sistema y la corrección de las transformaciones geométricas sobre el tablero táctico.

En lo relativo a la evaluación del modelo, el entrenamiento se ha llevado a cabo utilizando tanto entornos de Google Colab como la plataforma Roboflow, aprovechando las fortalezas analíticas de cada una. Para determinar la fiabilidad de la red neuronal, nos hemos regido estrictamente por los estándares de la industria en visión por computador. A continuación, se detallan las métricas clave seleccionadas para validar el éxito del entrenamiento:

- **mAP_0.5 (Mean Average Precision al 50%):** Constituye el estándar de referencia para medir la robustez de la detección. Evalúa si el recuadro (bounding box) predicho por la IA se superpone al menos en un 50% con el objeto real. En este contexto, se considera que un valor situado entre el 50% y el 79% es aceptable, mientras que superar la barrera del 80% indica un modelo excelente y fiable.
- **mAP_0.5:0.95**: Es una métrica mucho más exigente que calcula el rendimiento promedio bajo criterios de solapamiento estricto (desde el 50% hasta el 95%). Debido a su rigurosidad, obtener cifras superiores al 60% es indicativo de un resultado sobresaliente, siendo el estándar habitual valores entre el 30% y el 59%.
- **Precisión (Calidad):** Responde a la pregunta: "De todas las detecciones que ha hecho el modelo, ¿cuántas eran reales?". Esta métrica es fundamental para controlar los falsos positivos. El objetivo es situarse en una horquilla entre el 75% y el 89%, aspirando a superar el 90% para considerar el sistema óptimo.

- **Recall (Cantidad):** Responde a: "De todos los objetos que había en la imagen, ¿cuántos ha sido capaz de encontrar?". Evalúa la capacidad del sistema para no omitir ningún objeto presente en la escena (falsos negativos). Al igual que con la precisión, se busca superar el 90% para asegurar que no se pierde información del juego.
- **val/box_loss:** Es una métrica de error que cuantifica la distancia en píxeles entre el recuadro predicho y el real. Al tratarse de una función de pérdida, el objetivo es que este valor sea lo más cercano posible a 0, indicando un ajuste espacial perfecto.

Cabe mencionar que, para este análisis, se han combinado los informes detallados en formato CSV generados por Google Colab, que ofrecen una visión global de la evolución del entrenamiento, con las herramientas visuales de Roboflow, que permiten desglosar el mAP por clases individuales y detectar problemas específicos en objetos concretos.
Finalmente, la validación funcional en escenarios reales se ha realizado mediante una metodología de inspección visual sistemática. Dado que en un entorno de fútbol sala amateur no se dispone de sensores externos que proporcionen una "verdad absoluta" (Ground Truth) para contrastar la posición exacta de los jugadores, la verificación de la proyección 2D se realiza de manera manual. Este proceso implica seleccionar puntos aleatorios del vídeo procesado y comprobar si la transformación homográfica ha situado al jugador en la coordenada correcta del tablero táctico, asegurando así que las matemáticas detrás de la proyección son coherentes con la realidad visual del partido.

5.1. Primera Prueba: Validación Piloto

Para iniciar la fase experimental, se planteó una primera prueba piloto concebida como una "prueba de concepto". El objetivo principal en este estadio no consistía en obtener el modelo definitivo, sino en validar la corrección del flujo de trabajo diseñado, antes de proceder a la creación de un dataset masivo. Por consiguiente, este primer entrenamiento se llevó a cabo utilizando un conjunto reducido de 70 imágenes, todas extraídas y etiquetadas manualmente del mismo vídeo, ejecutando el proceso durante 100 epochs para evaluar la respuesta de la red neuronal.

Lo primero que destaca al analizar los resultados es la rapidez de convergencia del modelo. A pesar de contar con un número limitado de ejemplos, la curva de aprendizaje se estabilizó prematuramente. En relación con la métrica principal, el **mAP_0.5**, se alcanzó un valor final de **0.808 un mAP_0.5:0.95 de 0.635**. Considerando que en el ámbito de la visión por computador se estima como "fiable" un valor superior a 0.8, este resultado inicial confirma que la arquitectura YOLO se adapta adecuadamente a la detección de jugadores, logrando captar las características visuales distintivas (morfología, postura, contraste) sin requerir un volumen masivo de datos en esta fase preliminar.

No obstante, el análisis revela un desequilibrio significativo entre la Precision y el Recall de las detecciones, lo cual define el comportamiento de este primer modelo. Por un lado, la **Precisión** ascendió hasta un **96.5%**, una cifra notablemente elevada. Esto indica un comportamiento prudente por parte del algoritmo: la tasa de falsos positivos es mínima, evitando la confusión de elementos del fondo (vallas publicitarias, público u objetos en pista) con jugadores, lo cual resulta crítico para la limpieza del tablero táctico posterior.

Por otro lado, el **Recall** (sensibilidad) se situó en un **80.9%**. Aunque es un dato aceptable, señala la principal limitación de esta primera iteración: la omisión de detecciones. Esto aun así tiene un sentido, ya que este resultado unifica el recall de todos los objetos, pero como se ve más adelante, objetos como el balón o los corners son elementos que cuesta ser detectados por ser elementos tan pequeños.

Ilustración 16. Visualización del lote de validación

Por último, las métricas de error confirman la estabilidad técnica del proceso de aprendizaje. El valor de **val/box_loss** descendió hasta **0.03**, lo que evidencia que, una vez detectado el objeto, el ajuste espacial de la caja delimitadora (*bounding box*) sobre el jugador es extremadamente preciso. En conclusión, esta primera fase experimental arrojó un modelo fiable y robusto dentro de su dominio de entrenamiento; sin embargo, resultaba imperativo verificar su comportamiento al exponerlo a entornos visuales inéditos para validar su capacidad de generalización.

Es al realizar la inferencia sobre un nuevo escenario cuando se hicieron evidentes las limitaciones del modelo. A pesar de la robustez mostrada inicialmente, el rendimiento descendió drásticamente al cambiar el entorno: aumentaron los falsos negativos (omisión de jugadores y de la estructura de la pista) y, lo que es más crítico, aparecieron falsos positivos derivados del "ruido" visual, confundiendo al público de la grada y a los suplentes con jugadores activos, dado que el dataset original carecía de estos elementos de fondo. Asimismo, esta prueba reveló un defecto en la definición de la clase midfield, la cual tendía a confundirse con otras marcas circulares presentes en el pavimento. Finalmente, esta fase sirvió para identificar una discrepancia estratégica en la configuración del entrenamiento: el sistema estaba operando bajo una arquitectura de Detección de Objetos clásica (Bounding Boxes), desaprovechando la información de las máscaras poligonales que ya se habían etiquetado, lo que motivó la migración definitiva hacia la Segmentación de Instancias (Instance Segmentation) para las siguientes iteraciones.

Ilustración 17. Vídeo procesado con el primer Modelo

5.2 Segundo y tercer entrenamiento en Roboflow

Los dos análisis siguientes se llevaron a cabo directamente dentro del entorno de Roboflow. El objetivo de estas pruebas no era obtener el modelo final de producción, sino generar modelos intermedios lo suficientemente robustos para asistir en el etiquetado

automático de nuevas imágenes y, al mismo tiempo, comprender qué clases específicas estaba aprendiendo la red con mayor facilidad y en cuáles estaba fallando.

El primero de estos entrenamientos en Roboflow se realizó sobre un dataset de 333 imágenes, combinando 250 imágenes de un entorno conocido con otras 83 provenientes de un dataset completamente distinto para aportar variedad. Este experimento ya incorporaba la segmentación de instancias (*Instance Segmentation*) y la nueva clase para el círculo central (*midfield*). Se configuró el entrenamiento a 350 épocas, observándose un punto de inflexión claro a partir de la época 250, donde el rendimiento mejoró notablemente. El **mAP_0.5** pasó de oscilar en torno al 0.73 a alcanzar un pico final de **0.823**. Respecto al resto de métricas, se obtuvo un **mAP_0.5:0.95** de 0.67, una **Precisión** del 78,5%, un **Recall** del 83,2% y un **val/box_loss** estabilizado en 0,5.

Además, tal y como se mencionó anteriormente, Roboflow permite desglosar el análisis por clases, lo cual arrojó conclusiones muy valiosas. La primera evidencia es que los elementos estáticos y de gran tamaño son los que la red aprende con mayor facilidad; mientras que la media global fue de 0,82, elementos estructurales como el *court* (pista) y las áreas alcanzaron un **99%**, rozando la perfección, e incluso el *midfield* llegó al **100%**. Estos resultados validaron la decisión de utilizar precisamente estos elementos (pista, áreas y medio campo) como puntos de anclaje para la homografía. Por contraposición, se observa que objetos pequeños como el balón (*ball*) o los córners presentan dificultades significativas de detección. Esto resulta lógico dado su tamaño en píxeles, y sugiere que la solución pasa por ampliar drásticamente el dataset o implementar algoritmos específicos de *tracking* temporal para el balón, priorizando el seguimiento de la trayectoria sobre la detección aislada frame a frame para evitar pérdidas.

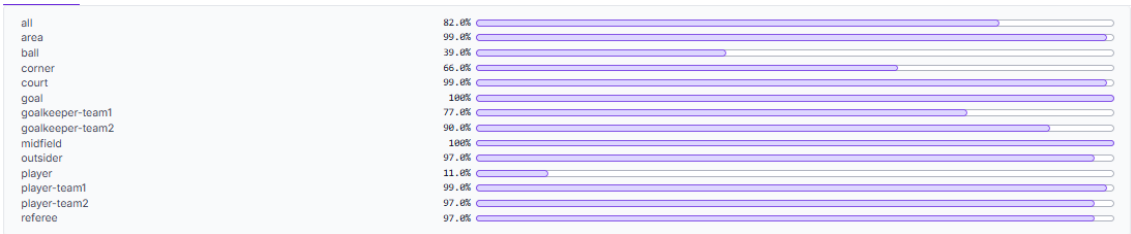

Ilustración 18. Análisis detallado por clase

Por otro lado, el análisis visual de las detecciones sobre el conjunto de test revela un comportamiento prometedor: los objetos no solo se identifican con un alto grado de confianza (*confidence score*), sino que las máscaras generadas se ajustan con gran fidelidad al contorno real de los elementos, superando las limitaciones espaciales propias de las *bounding boxes* tradicionales.

Ilustración 19.Visualización del lote de test

El segundo entrenamiento en Roboflow se diseñó para validar la capacidad de adaptación del sistema a nuevos escenarios. Para ello, se conformó un dataset híbrido de 525 imágenes, integrando 250 muestras del primer entorno, 250 del segundo y añadiendo un conjunto crítico de 25 imágenes extraídas específicamente del vídeo utilizado para el desarrollo del MVP. En esta ocasión, la estrategia de entrenamiento difirió de la anterior: en lugar de iniciar el proceso desde cero (*tabula rasa*), se aplicó la técnica de **Fine-Tuning** (ajuste fino), cargando los pesos pre-entrenados del modelo previo para aprovechar el conocimiento ya adquirido. Esta metodología permitió reducir la carga computacional a solo 100 épocas, punto en el que se activó el criterio de parada temprana (*Early Stopping*) al no observarse mejoras significativas en la función de pérdida, evitando así el sobreajuste.

Los resultados de esta segunda iteración superaron métricamente a la prueba anterior, alcanzando un **mAP_0.5** global de **0.886** y un **mAP_0.5:0.95** de **0.70**. Asimismo, las métricas de calidad y cantidad experimentaron un ascenso notable, situándose la **Precisión** en un 88,6% y el **Recall** en un 86,7%, con un valor de **val/box_loss** estabilizado en torno a 0,67. Desglosando el rendimiento por clases, se observa una robustez generalizada con detecciones superiores al 90% en la mayoría de los elementos. Mención especial merece la clase *balón* (*ball*), que, a pesar de seguir siendo el elemento más complejo por su reducido tamaño, experimentó una mejora sustancial hasta alcanzar el 71%. Este dato confirma la hipótesis de que, al incrementar la variabilidad del dataset, la red neuronal refina su capacidad para extraer características visuales sutiles en objetos pequeños.

En conclusión, este experimento valida empíricamente la estrategia de escalado del dataset: la incorporación de un número reducido de muestras específicas (apenas 25 imágenes del nuevo entorno) resulta suficiente para que el modelo generalice y se adapte con éxito al nuevo contexto visual, mejorando sus métricas globales sin necesidad de reetiquetar miles de fotogramas.

5.3. Entrenamiento completo en Google Colab

En esta fase final, la experimentación se trasladó a Google Colab para tener un control total sobre los hiperparámetros. El proceso se dividió en dos etapas secuenciales: un primer entrenamiento con el dataset base de 500 imágenes (la combinación de las 250 del primer entorno más las 250 del segundo) y un posterior refinamiento (*Fine-Tuning*) incorporando las 25 imágenes específicas del nuevo dataset.

Es fundamental aclarar que no resulta viable establecer una comparación directa entre el número de épocas ejecutadas en Google Colab y las de Roboflow. Mientras que en Roboflow los parámetros se ajustan de manera automatizada (caja negra), en Google Colab se definieron tasas de aprendizaje (*learning rates*) bajas y suaves manualmente, permitiendo una convergencia más estable y eficiente del modelo. Debido a esto, aunque se limitó el entrenamiento a 100 épocas para el primer modelo y 50 para el segundo —condicionado también por las restricciones de tiempo de uso de la GPU gratuita de Google (aprox. 4 horas)—, la calidad del aprendizaje no es inferior a las 350 épocas automatizadas de la plataforma anterior.

Asimismo, es importante destacar un cambio cualitativo en la evaluación: al consolidarse el proyecto como un sistema de **Segmentación de Instancias**, el análisis ya no se limita a las cajas delimitadoras (*bounding boxes*), sino que incorpora las métricas de **Máscara (M)**. El concepto de máscara hace referencia a la silueta exacta píxel a píxel del objeto detectado, lo que supone un nivel de exigencia y detalle muy superior al de una simple caja rectangular. Por tanto, los resultados presentados a continuación desglosan el rendimiento en ambas vertientes: Boxes (B) y Máscaras (M).

En el primer análisis sobre las 500 imágenes base, los resultados de detección (*bounding boxes*) mostraron un salto de calidad significativo respecto al modelo inicial de 70 imágenes. Se alcanzó un **mAP@0.5 (B)** de **0.902**, con un **mAP@0.5:0.95 (B)** de 0.709, una **Precisión (B)** del 92.5%, un **Recall (B)** del 86.1% y un **val/box_loss (B)** estabilizado en 0.635. Estas cifras confirman una gran robustez en la localización de los jugadores.

No obstante, dado que el objetivo es sentar una base sólida para futuros desarrollos de proyección 2D, es crítico analizar el rendimiento de la segmentación. En este apartado, las métricas de máscara arrojaron un **mAP_0.5 (M)** de **0.773**, un **mAP_0.5:0.95 (M)** de 0.481, una **Precisión (M)** del 80%, un **Recall (M)** del 77.8% y un **val/seg_loss (M)** de 1.167. Aunque estos datos se consideran notablemente buenos para una primera aproximación a la segmentación, la experiencia previa nos indica que un buen rendimiento sobre el set de validación no garantiza, por sí solo, una generalización perfecta ante imágenes inéditas, por lo que fue necesario proceder a la siguiente fase de refinamiento.

Tras someter el vídeo de prueba al análisis con el modelo intermedio, se observó un comportamiento mixto. Si bien la red neuronal comenzaba a mostrar una incipiente capacidad de generalización —intuyendo la estructura de la pista (*court*) y logrando distinguir a los jugadores respecto al público de la grada—, la detección global distaba mucho de ser aceptable para un producto final. Los fallos eran claramente atribuibles al cambio de dominio visual (*Domain Shift*): el modelo, al no haber visto nunca ese pabellón específico, fallaba al procesar el color concreto de las líneas, el tono del área o la tipología del balón utilizado.

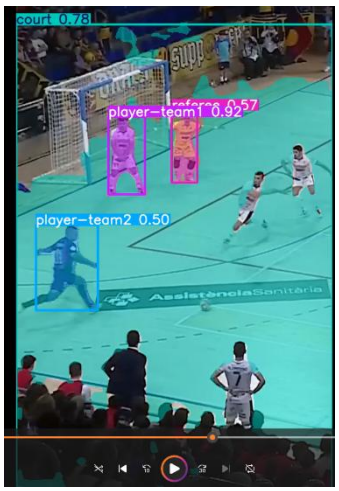

Ilustración 20. Vídeo procesado con el Segundo Modelo

Para solventar esta brecha, se ejecutó un último entrenamiento incorporando las 25 imágenes etiquetadas del nuevo entorno mediante una estrategia de *Fine-Tuning*. Los resultados numéricos confirmaron la hipótesis: la mejora en las **Bounding Boxes (B)** fue marginal —dado que la localización ya era buena—, alcanzando un **mAP_0.5 (B)** de **0.918** y una **Precisión (B)** del 94%. Sin embargo, el verdadero salto cualitativo se produjo

en la **Segmentación (Máscaras)**. Las métricas de silueta experimentaron un crecimiento drástico: el **mAP_0.5 (M)** subió hasta **0.845** (una mejora relativa del **+7,55%**), y la métrica de alta exigencia **mAP_0.5:0.95 (M)** alcanzó el **0.563** (**+13,03%**). El dato más revelador fue la **Precisión (M)**, que escaló hasta un **0.939** (**+16,15%**), lo que indica matemáticamente que el recorte de la figura del jugador se volvió mucho más limpio y exacto.

Finalmente, la prueba de fuego consistió en procesar el vídeo completo con este modelo final. Las detecciones resultaron muy satisfactorias: las máscaras se ajustaban fielmente al contorno biomecánico de los jugadores y los elementos estructurales de la pista se identificaban con precisión, discriminando correctamente a los jugadores activos de los *outsiders*. No obstante, desde una perspectiva de honestidad ingenieril, es necesario reconocer ciertas limitaciones residuales: en momentos puntuales se producen "dobles detecciones" (clasificando a un mismo sujeto como jugador y árbitro simultáneamente) y persisten dificultades con los porteros debido a la singularidad de sus equipaciones. A pesar de estos casos extremos (*edge cases*), la calidad general de la inferencia se consideró idónea para alimentar la fase crítica del proyecto: la transformación y proyección al tablero táctico.

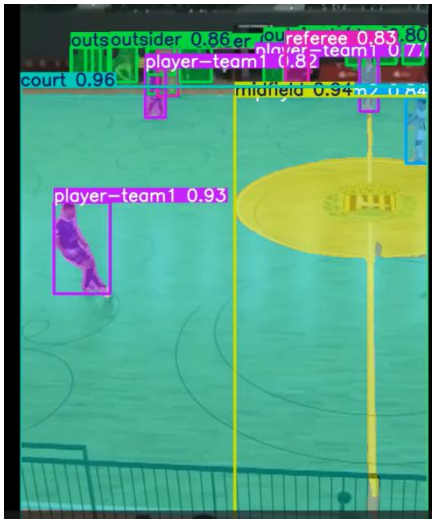

Ilustración 21. Vídeo procesado con el Tercer Modelo

5.4. Transformación al Tablero y Validación Geométrica

Como culminación del flujo de trabajo, el módulo de transformación al tablero se nutre directamente de la calidad de las detecciones del apartado anterior. Es evidente que la precisión geométrica depende de una detección limpia; sin embargo, el tratamiento de los falsos positivos o las inconsistencias de identidad ("dobles detecciones") corresponde a la lógica de negocio del seguimiento de jugadores (que se abordará completamente en desarrollos futuros más allá del *Frame 0*). El objetivo de este apartado es validar

específicamente la **fidelidad matemática de la proyección**: verificar si los jugadores detectados en el vídeo acaban situados en la coordenada métrica correcta del tablero táctico 2D.

Es importante recordar el contexto de este MVP: operamos bajo ciertas simplificaciones geométricas conscientes, como la asunción de una posición de cámara fija en el eje horizontal (X=300) y un rango de apertura de lente (FOV) limitado entre 30º y 60º. Estas restricciones, necesarias para la viabilidad del prototipo, pueden introducir ligeras desviaciones que analizaremos a continuación. Para validar el sistema, se ha establecido una metodología de muestreo temporal, analizando la salida del algoritmo cada 5 segundos en un clip de 29 segundos.

5.4.1 El "Golden Standard": Frame 0

El punto de partida es la inicialización o **Frame 0** (mostrado en la Ilustración 15). En este instante estático, la transformación es prácticamente perfecta. Todos los jugadores son ubicados en su posición exacta en el tablero, validando que la lógica de proyección funciona al 100% cuando las condiciones son ideales. Esto demuestra que las ecuaciones base de la homografía son correctas.

5.4.2 Análisis Dinámico Temporal

A continuación, se presenta la evolución de la transformación a lo largo del clip de vídeo de 29 segundos, analizando fotogramas clave cada 5 segundos para verificar la estabilidad de la proyección.

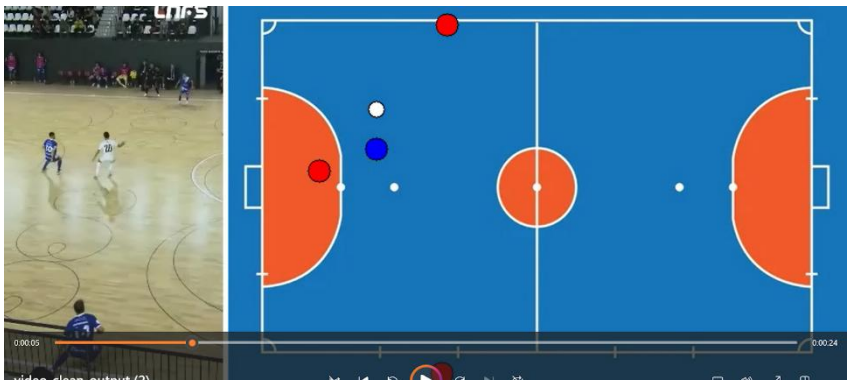

Ilustración 22. Imagen al segundo 5

Segundos 5: En este primer instante, la fase de detección opera con gran precisión, identificando correctamente a los tres jugadores del equipo azul (puntos rojos) y al jugador del equipo blanco (punto azul). Un aspecto funcional destacable es la gestión del balón: aunque visualmente se encuentra parcialmente oculto tras el jugador blanco, el sistema mantiene su posicionamiento gracias a la lógica de interpolación temporal, evitando la pérdida de datos. No obstante, al analizar la proyección en el tablero, se aprecia una ligera desviación lateral hacia la izquierda en la disposición de los jugadores respecto a su posición real en la pista, consecuencia directa de la simplificación del ángulo de cámara en los parámetros de la homografía.

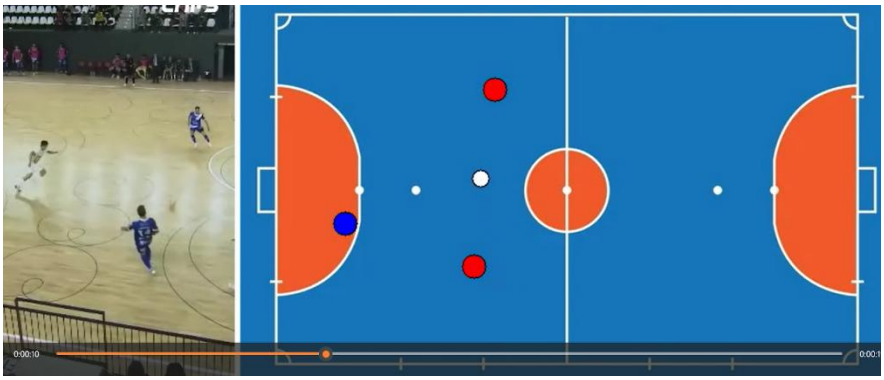

Ilustración 23. Imagen al segundo 10

Segundo 10: Al avanzar al segundo 10, se confirma la tendencia observada anteriormente. Las máscaras siguen siendo correctas, sin falsos positivos, lo que valida la robustez del modelo YOLOv8. Sin embargo, el error de proyección se hace más evidente en los jugadores situados en el extremo izquierdo de la imagen; en el tablero táctico, estos aparecen posicionados más cerca del área de lo que realmente están. Esto corrobora que la calibración estática de la cámara (X=300) introduce una deriva sistemática en los bordes del encuadre.

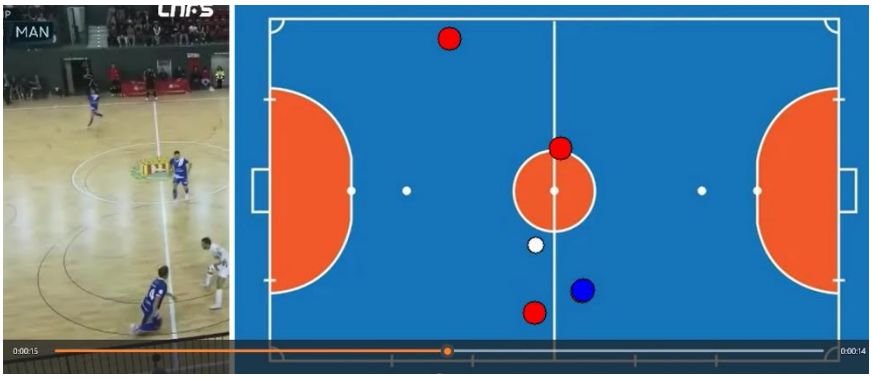

Ilustración 24. Imagen al segundo 15

Segundo 15: En este fotograma el sistema se enfrenta a una mayor complejidad escénica. Se observa una inconsistencia puntual en la detección: el jugador del equipo blanco recibe una doble etiqueta (detectado simultáneamente como azul y blanco), un fallo que deberá corregirse en la lógica de negocio futura. En cuanto a la geometría, se hace patente el reto de la profundidad: mientras que los dos jugadores situados en la parte inferior de la imagen (más cerca de la lente) se proyectan con precisión milimétrica, el jugador situado cerca del círculo central aparece representado en el tablero mucho más lejos de su posición real, evidenciando la necesidad de incorporar referencias visuales adicionales para corregir la coordenada Y en largas distancias.

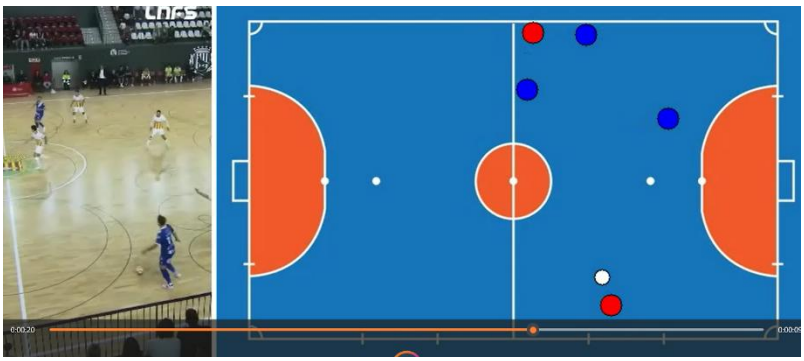

Ilustración 25. Imagen al segundo 20

Segundo 20: En el segundo 20, el sistema recupera la estabilidad en el conteo de jugadores. La disposición táctica general en el tablero refleja fielmente la formación del equipo en la pista, lo que valida la utilidad del MVP para el análisis de estrategias básicas. Sin embargo, persiste el desplazamiento en el eje vertical (Y) mencionado anteriormente: los elementos tienden a proyectarse con una "profundidad exagerada", alejándose del

punto de vista de la cámara más de lo necesario debido a la falta de un segundo punto de anclaje (como el *midfield*) en la ecuación de transformación actual.

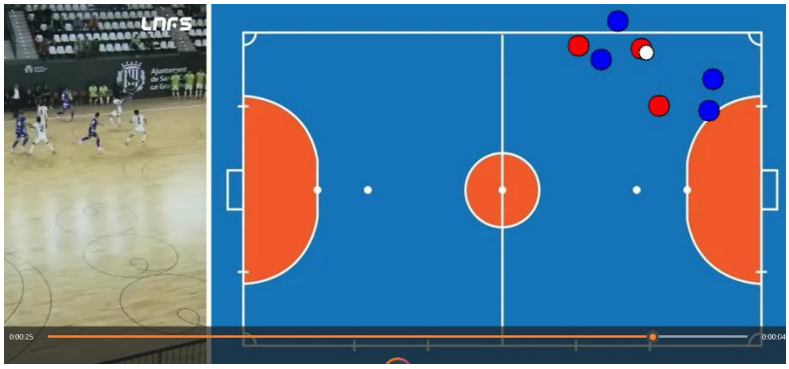

Ilustración 26. Imagen al segundo 25

Segundo 25: Este instante representa el escenario más crítico del análisis, donde convergen los márgenes de error de los ejes X e Y. La suma del desplazamiento lateral y la imprecisión en profundidad provoca que el jugador del equipo blanco sea proyectado matemáticamente fuera de los límites del terreno de juego en el tablero virtual. Este "edge case" es fundamental, ya que delimita el rango de operación seguro del algoritmo actual y subraya la necesidad de refinar el cálculo del FOV (Campo de Visión) para evitar que los jugadores en los límites de la imagen se salgan del mapa.

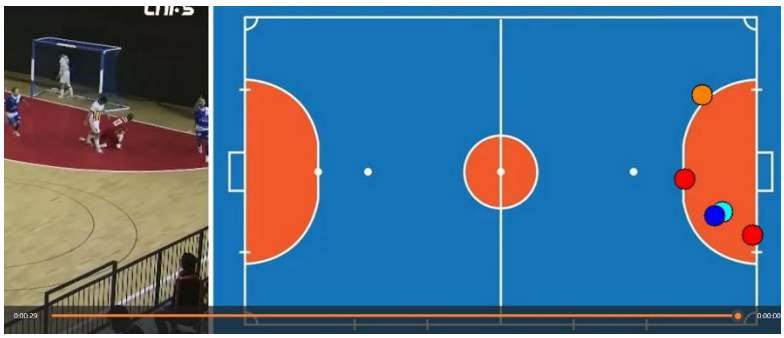

Ilustración 27. Imagen al segundo 29

Segundo 29 (Final): El último fotograma ofrece un resumen de las capacidades y limitaciones del prototipo. Por un lado, se detecta un falso positivo en la portería (identificando al portero azul erróneamente), y el sistema no logra proyectar al jugador que ha salido físicamente de los límites de la pista, lo cual entra dentro de las restricciones de diseño. Por otro lado, este frame demuestra la eficacia de la lógica de "Anclajes de

Área": los jugadores que se encuentran dentro o cerca del área se proyectan con una precisión casi absoluta en el tablero, demostrando que el uso de las líneas del área como referencia geométrica corrige eficazmente la perspectiva cuando los sujetos están en esa zona.

5.4.3 Conclusión de la Validación

El análisis demuestra que el sistema cumple sobradamente su función como MVP: **la traducción de la realidad 3D a un plano 2D es funcional y coherente tácticamente.** Las desviaciones observadas (desplazamientos laterales y errores de profundidad en larga distancia) no son fallos del algoritmo, sino consecuencias directas de las simplificaciones de parámetros (FOV de 30º y posición de cámara fija). La solución técnica para la versión 2.0 es clara: implementar una lógica que utilice el *midfield* como segundo punto de anclaje para corregir la coordenada Y y, posiblemente, reducir el ángulo de FOV estimado (inferior a 30º) para minimizar la deriva lateral.

En definitiva, disponemos de una base sólida. El sistema "entiende" el juego y, con el refinamiento de los parámetros de cámara y la introducción de un suavizado de trayectorias (para evitar saltos bruscos), estará listo para un despliegue productivo.

6. Planificación

Para afrontar un proyecto de esta envergadura, he considerado que la organización era tan vital como el propio desarrollo técnico. Esta metodología de trabajo la he adoptado directamente de mi experiencia durante las prácticas en el grupo L'Oréal, donde aprendí que la eficiencia depende del orden. Allí fui el encargado de diseñar una aplicación en Power Apps para automatizar la creación de diagramas de Gantt dinámicos, y he querido aplicar esa misma filosofía aquí.

Gracias a esta herramienta, he podido visualizar las tareas por semanas de manera dinámica, asignar prioridades mediante códigos de color y, lo más importante, vincular

directamente los recursos a cada tarea. Esto me permitía, por ejemplo, tener en el propio calendario los accesos directos a los *notebooks* de Google Colab o a las versiones específicas del dataset en Roboflow, agilizando muchísimo el flujo de trabajo diario. Tal y como se muestra en la Ilustración 31 incorporada en el Anexo, la interfaz cuenta con una línea roja vertical que indica el día actual en tiempo real; esto permite evaluar el progreso de un solo vistazo y mover las tareas a lo largo del calendario dinámicamente según se van completando o reprogramando.

El proyecto ha abarcado desde el 1 de septiembre hasta el 12 de febrero y se ha estructurado en cinco fases principales que, en algunos momentos, se han ido solapando para optimizar tiempos.

Las dos primeras semanas de septiembre (del 1 al 14) las dediqué exclusivamente a la **Fase de Análisis.** Fue el momento de investigar el estado del arte, ver qué soluciones de tracking existían y definir el stack tecnológico, decantándome finalmente por YOLOv8.

A partir del 15 de septiembre y hasta el 1 de noviembre, entré en **la Fase de Entrenamiento y Datos**. Fue la etapa más larga y laboriosa. Empecé recopilando los vídeos, y mientras iba etiquetando manualmente en Roboflow, ya iba lanzando las primeras pruebas de concepto en Google Colab. Esto me permitió ir corrigiendo la forma de etiquetar sobre la marcha sin esperar al final.

Una vez validado el modelo de detección, del 1 de noviembre al 19 de diciembre abordé la **Fase de Proyección Geométrica**. Esta etapa fue el núcleo matemático del TFG. Al comprobar que las homografías tradicionales fallaban, dediqué este tiempo a desarrollar un sistema propio de tres pasos: primero, la Calibración Estática para localizar la posición física de la cámara mediante triangulación estadística; segundo, la Proyección Dinámica, calculando el ángulo de rotación frame a frame asumiendo un FOV fijo; y finalmente, el Mapeo de Entidades, donde implementé los algoritmos de "Escáner Vertical" y "Anclajes" para traducir la posición de los píxeles al tablero 2D con precisión métrica.

Aprovechando el periodo navideño, del 19 de diciembre al 8 de enero, abordé la Fase de **Lógica de Frame 0**. Fue el momento de "limpiar" los datos: implementé los filtros de color para separar equipos y árbitros, y programé la restricción lógica para asegurar que el sistema solo detectase a un máximo de 10 jugadores de campo, eliminando el ruido.

Finalmente, desde el 8 de enero hasta el 12 de febrero, me he dedicado a la **Fase de Documentación**. He recopilado todos los CSV de resultados, he generado las gráficas comparativas y he redactado esta memoria. Cabe mencionar que la defensa del TFG no se incluye en este cronograma, ya que tendrá lugar en una fecha posterior.

Durante todo este tiempo, he mantenido un **seguimiento mensual** con mi tutor. La dinámica era sencilla: a principios de cada mes (septiembre, octubre, noviembre, diciembre, enero y febrero) acordábamos una reunión con una semana de antelación para revisar los hitos cumplidos y planificar el siguiente bloque de trabajo.

	A	B	C	D
1				
2				
3		Number of task shown:	15	
4	TFG Informática	Week of:	lunes, 1 de septiembre de 2025	
5	Jaime Ballesteros Calvo	Display	-23	
6				
7	TASKS	NOTES	START	END
9	**Análisis y Herramientas**			
10	Investigación Estado del Arte		01/09/2025	07/09/2025
11	Selección de Stack Tecnológico		08/09/2025	14/09/2025
12	**Entrenamiento y Datos**			
13	Recopilación de Vídeos		15/09/2025	20/09/2025
14	Etiquetado en Roboflow		21/09/2025	01/11/2025
15	Configuración Entorno Colab		25/09/2025	30/09/2025
16	Entrenamiento Iterativos		01/10/2025	01/11/2025
17	**Proyección Geométrica y Matemáticas**			
18	Algoritmo de Calibración Estática (Cámara)		01/11/2025	15/11/2025
19	Cálculo de FOV y Rotación Dinámica		16/11/2025	30/11/2025
20	Mapeo Híbrido		01/12/2025	19/12/2025
21	Interpolación de Trayectoria del Balón		15/12/2025	19/12/2025
22	**Lógica de Frame 0**			
23	Lógica de Inicialización		20/12/2025	08/01/2026
24	**Documentación**			
25	Redacción de Memoria		09/01/2026	10/02/2026
26	Extracción de Resultados y Gráficas		02/02/2026	08/02/2026
27	Revisión Final y Formato		11/02/2026	12/02/2026
28	**Reuniones de seguimiento**			
29	Revisiones		01/09/2025	08/09/2025
30	Revisiones		01/10/2025	07/10/2025
31	Revisiones		01/11/2025	07/11/2025
32	Revisiones		01/12/2025	08/12/2025
33	Revisiones		01/01/2026	07/01/2026
34	Revisiones		01/02/2026	09/02/2026

Ilustración 28. Gantt Chart

7. Presupuesto

El presente apartado detalla el estudio económico asociado al desarrollo del proyecto **FutsalVision**. Para la elaboración del presupuesto, se ha simulado un escenario empresarial real, considerando los costes que tendría la ejecución de este sistema en el mercado laboral actual.

El presupuesto se desglosa en tres partidas fundamentales: costes de personal (asignando roles profesionales diferenciados según la naturaleza de las tareas), amortización de recursos hardware y costes operativos de licencias y servicios en la nube.

7.1 Costes de Personal

Dado que el desarrollo de un sistema de visión artificial requiere competencias multidisciplinares, se ha modelado el equipo de trabajo dividiendo la carga total de horas (estimada en 350 horas) en tres perfiles técnicos diferenciados. Aunque en el ámbito académico todas las tareas han sido ejecutadas por el alumno, en un entorno corporativo se distribuirían de la siguiente manera:

- **Jefe de Proyecto:** Responsable de la planificación (Gantt), gestión de recursos, interlocución con el tutor/cliente y redacción de la documentación final.
- **Ingeniero de IA y Datos:** Encargado de la gestión del dataset en Roboflow, configuración de los entrenamientos en Google Colab y validación de métricas.
- **Desarrollador de Software:** Responsable de la implementación de los algoritmos de homografía, lógica de Frame 0 y procesamiento de vídeo con OpenCV.

Rol Profesional	**Descripción de Tareas**	**Precio Hora (€/h)**	**Horas Imputadas**	**Coste Total (€)**
Jefe de Proyecto	Planificación, gestión, redacción memoria.	45,00 €	60 h	2.700,00 €
Ingeniero de IA	Etiquetado, entrenamientos YOLO, Fine-Tuning.	30,00 €	140 h	4.200,00 €
Desarrollador Software	Algoritmos, Homografía, Lógica Python.	25,00 €	150 h	3.750,00 €

TOTAL PERSONAL			**350 h**	**10.650,00 €**

Tabla 1. Coste Personal

7.2. Amortización de Hardware

Para el desarrollo del proyecto se ha requerido un equipo informático de alto rendimiento capaz de gestionar previsualizaciones de vídeo y tareas de programación. No se imputa el coste total del equipo, sino la amortización lineal correspondiente a los **5 meses** de duración del proyecto (septiembre - febrero), asumiendo una vida útil estándar de 4 años para equipos tecnológicos.

- *Equipo Base:* Portátil con procesador i7/Ryzen 7 y GPU dedicada (Valor ref: 1.500 €).
- *Fórmula:* (Valor/ 48 meses) * 5 meses

Recurso Hardware	**Valor de Mercado**	**Vida Útil**	**Uso en Proyecto**	**Amortización (€)**
Laptop Workstation	1.500,00 €	4 años	5 meses	156,25 €
Periféricos (Monitor 4K, Accesorios)	300,00 €	4 años	5 meses	31,25 €
TOTAL HARDWARE				**187,50 €**

Tabla 2. Amortización de Hardware

7.3. Costes de Software y Servicios Cloud

Esta partida recoge los costes derivados del uso de plataformas SaaS (*Software as a Service*) y potencia de computación en la nube. Se han valorado los precios de mercado

de las licencias necesarias para replicar el entorno de desarrollo fuera del ámbito académico gratuito.

- **Google Colab Pro:** Necesario para acceder a GPUs de alto rendimiento (A100/V100) y tiempos de ejecución extendidos para los entrenamientos de YOLOv8.
- **Roboflow:** Se presupuesta la licencia profesional básica para permitir la generación de versiones del dataset y el uso de la API de despliegue.

Concepto	**Tipo de Servicio**	**Coste Mensual (€)**	**Duración**	**Coste Total (€)**
Google Colab Pro	Computación GPU Cloud	10,79 €	3 meses	32,37 €
Roboflow Starter	Gestión Dataset y Hosting	99,00 €*	3 meses	297,00 €
Entorno Python	Librerías Open Source	0,00 €	-	0,00 €
TOTAL SOFTWARE				**619,37 €**

Tabla 3. Costes de Software y Servicios Cloud

**Nota: Aunque se ha utilizado la licencia académica gratuita de Roboflow, se incluye el coste del plan "Starter" ($99/mes aprox) para reflejar el valor real de mercado de la herramienta utilizada.*

7.4. Resumen y Presupuesto Final

Finalmente, se presenta el coste total agregado del proyecto, incluyendo un porcentaje de costes indirectos (electricidad, conectividad, gastos generales) estimado en un 10% sobre el subtotal.

Capítulo	Importe (€)
Costes de Personal	10.650,00 €
Amortización de Hardware	187,50 €
Licencias y Software	619,37 €
Subtotal Ejecución Material	**11.559,87 €**
Costes Indirectos (10%)	1.155,99 €
Base Imponible	**12.715,86 €**
IVA (21%)	2.670,33 €
PRESUPUESTO TOTAL	**15.283,19 €**

Tabla 4. Presupuesto Final

El presupuesto total para la ejecución del proyecto **FutsalVision** asciende a la cantidad de **quince mil dosciento ochenta y seis euros con diecinueve céntimos**. Esta valoración refleja la inversión necesaria para desarrollar un Producto Mínimo Viable (MVP) de análisis deportivo automatizado en un contexto profesional.

8. Marco Legal y Propiedad Intelectual

El desarrollo de un sistema de inteligencia artificial que procesa imágenes de personas en movimiento conlleva una responsabilidad legal significativa. Este proyecto se enmarca dentro de la legislación española y europea, abordando dos vertientes principales: la protección de datos personales y el cumplimiento de las licencias de software utilizadas.

8.1. Protección de Datos y Privacidad (RGPD)

El Reglamento General de Protección de Datos (RGPD) clasifica las imágenes faciales como datos biométricos cuando se utilizan para identificar a una persona de forma unívoca.

- **Tratamiento de Imágenes:** Aunque el objetivo de *FutsalVision* es el análisis táctico (movimiento de cuerpos) y no el reconocimiento facial, el sistema procesa fotogramas donde los rostros pueden ser visibles. Para este TFG, el uso de estas imágenes se ampara en la **excepción académica y de investigación**. Sin embargo, en una hipotética explotación comercial, el sistema debería implementar obligatoriamente una técnica de *Privacy by Design* (Privacidad desde el Diseño), aplicando un difuminado automático (*blurring*) de los rostros antes de cualquier procesamiento en la nube, anonimizando así el dato.
- **Protección del Menor:** El fútbol sala base implica, mayoritariamente, la grabación de menores de edad. Según la Ley Orgánica de Protección de Datos (LOPD-GDD), el tratamiento de imágenes de menores de 14 años (Agencia Española de Protección de Datos [AEPD], 2018) requiere el consentimiento expreso de padres o tutores legales. Actualmente, el dataset utilizado proviene de fuentes públicas o grabaciones propias con consentimiento académico, pero un despliegue real exigiría que los clubes firmaran cesiones de derechos de imagen específicas que autoricen el procesamiento algorítmico con fines deportivos.

8.2. Licenciamiento de Software y Limitaciones Comerciales

La viabilidad del proyecto depende del cumplimiento de los términos de uso de las herramientas de terceros empleadas. Es crucial distinguir entre el uso académico actual y las restricciones para un producto comercial.

1. **Modelo YOLOv8 (Ultralytics):** Esta es la limitación legal más importante. YOLOv8 se distribuye bajo licencia **AGPL-3.0 (Ultralytics, s.f.)**. Esto significa que es gratuito para investigación y uso académico, pero si se quisiera comercializar el software, la licencia obliga a liberar el código fuente de toda la aplicación (*Open Source*). Para vender *FutsalVision* como un producto privado

(SaaS) sin liberar el código, sería necesario adquirir una licencia **Enterprise** de Ultralytics.

2. **Roboflow:** Para este proyecto se ha utilizado el plan *Public* (gratuito para estudiantes). La contrapartida legal de esta licencia es que el dataset generado debe ser **público y de código abierto** (CC BY 4.0) (Roboflow, s.f.). Si se quisiera privatizar los datos para que sean un activo exclusivo de la empresa, sería obligatorio migrar a un plan de pago comercial que garantice la propiedad privada de las imágenes.
3. **Google Colab:** Los términos de servicio de Google prohíben el uso de su capa gratuita para minería de criptomonedas o ataques informáticos, pero permiten la investigación. No obstante, para un entorno de producción comercial, no es una plataforma legalmente viable debido a la falta de garantías de servicio (SLA). La solución pasaría por desplegar el modelo en servidores propios o instancias cloud dedicadas (AWS/GCP) bajo contratos comerciales.

9. Entorno Socioeconómico e Impacto

La justificación de este proyecto trasciende lo puramente técnico. Tal y como se desprende de la investigación de mercado realizada en el Trabajo de Fin de Grado de Administración y Dirección de Empresas (ADE) vinculado a esta ingeniería, existe una brecha digital crítica en el deporte no profesional que este sistema pretende cerrar.

9.1. Democratización del Dato Deportivo (Mercado)

Actualmente, el mercado del análisis deportivo presenta una estructura polarizada. En la cúspide, entidades de élite invierten millones de euros en departamentos de I+D; un ejemplo claro es el **Barça Innovation Hub**, que financia startups de *SportsTech* para monitorizar cargas físicas y tácticas. Sin embargo, el 99% de los clubes (la base de la pirámide) opera en un "océano azul" desatendido: no tienen presupuesto para sistemas como *Catapult* o *Wimu*, ni infraestructura para instalar cámaras fijas en pabellones municipales.

FutsalVision impacta directamente en este nicho de mercado desaprovechado. Al sustituir hardware de miles de euros por software inteligente que funciona con cualquier vídeo de móvil, se reduce la barrera de entrada económica. Esto valida el proyecto no solo como una ingeniería viable, sino como un producto con un **Coste de Adquisición de Cliente (CAC)** potencialmente bajo y una escalabilidad masiva, atacando la "Long Tail" del deporte amateur.

9.2. Alineación con la Agenda 2030 (ODS)

Desde una perspectiva social, el proyecto se alinea con los Objetivos de Desarrollo Sostenible (ODS) de las Naciones Unidas (Naciones Unidas, 2015), promoviendo la igualdad de oportunidades a través de la tecnología:

- **ODS 10: Reducción de las Desigualdades.** La tecnología actúa como un ecualizador. Hasta ahora, un equipo de barrio no podía competir en análisis táctico con una academia profesional. Al democratizar estas herramientas, se reducen las desigualdades competitivas derivadas de la falta de recursos económicos, permitiendo que el talento (jugadores y entrenadores) destaque por su mérito y no por su presupuesto.
- **ODS 9: Industria, Innovación e Infraestructura.** El proyecto fomenta la digitalización de clubes modestos que, tradicionalmente, han permanecido analógicos. Introducir cultura del dato e inteligencia artificial en la gestión deportiva base moderniza el tejido asociativo local.
- **ODS 3: Salud y Bienestar.** Aunque el enfoque es táctico, la monitorización del movimiento es el primer paso para prevenir lesiones. Poner herramientas de seguimiento al alcance de entrenadores amateur contribuye a una práctica deportiva más segura y controlada para los deportistas federados.

En conclusión, este trabajo demuestra que la visión artificial avanzada no tiene por qué ser un lujo exclusivo de la élite, sino una palanca de transformación social y económica para el deporte base.

10. Conclusiones y Pasos Futuros

La realización de este Trabajo de Fin de Grado ha supuesto mucho más que la simple implementación de un algoritmo de visión por computador; ha constituido la validación técnica de una hipótesis de mercado. Al inicio de este proyecto, se planteó la pregunta de si era posible democratizar el análisis táctico deportivo, llevando herramientas propias de la élite a los clubes modestos utilizando únicamente una cámara de teléfono móvil. Tras meses de desarrollo, entrenamiento y pruebas matemáticas, la respuesta es un rotundo sí. Desde una perspectiva estrictamente técnica, los resultados obtenidos superan las expectativas iniciales para un entorno tan caótico como el fútbol sala amateur. La elección de la arquitectura **YOLOv8** y el proceso de refinamiento iterativo (*Fine-Tuning*) han demostrado ser la estrategia correcta, alcanzando una precisión de detección (**mAP@0.5**) del **91.8%** y una Precisión en máscaras del **94%**. Estos datos confirman que el modelo no solo "ve" el juego, sino que comprende la morfología de los jugadores con la robustez necesaria para generar datos fiables.

Sin embargo, el mayor hallazgo de este trabajo no reside en la detección, sino en la proyección. La investigación ha revelado que las soluciones matemáticas estándar ("homografías de libro") fallan estrepitosamente ante la realidad de una grabación manual con zoom y movimiento. La implementación del **"Proyector Geométrico Híbrido"** (combinando el escáner vertical y los anclajes de área) ha sido el verdadero punto de inflexión del proyecto, permitiendo traducir coordenadas de vídeo a un tablero táctico 2D con una coherencia métrica que parecía inalcanzable sin sensores externos.

A nivel personal y de gestión, este proyecto ha sido un ejercicio de madurez profesional. La aplicación de metodologías ágiles y herramientas de planificación —inspiradas en mi experiencia laboral en L'Oréal— ha sido tan crítica como el propio código Python. He aprendido que en un proyecto de ingeniería, la capacidad para organizar los recursos, gestionar los tiempos de entrenamiento en la nube y anticipar los cuellos de botella (como el etiquetado manual) es lo que marca la diferencia entre una idea teórica y un producto funcional.

En definitiva, *FutsalVision* deja de ser una idea abstracta para convertirse en un prototipo funcional (MVP) que sienta las bases para una transformación real en el deporte base. No hemos solucionado el fútbol sala, pero se ha demostrado que se puede digitalizar.

10.1 Líneas futuras

Aunque los objetivos académicos de este Trabajo de Fin de Grado se han alcanzado satisfactoriamente con la validación de un MVP funcional, mi visión para FutsalVision trasciende el ámbito universitario. Concibo este proyecto no como un ejercicio final, sino como el punto de partida para una solución de mercado real y escalable. Soy consciente de que transformar un prototipo de investigación en un producto robusto requiere un salto cualitativo en la ingeniería y recursos que superan la capacidad individual, por lo que la evolución natural del proyecto pasará por abrir el desarrollo a la colaboración con otros compañeros para escalar la arquitectura. A partir de esta base consolidada, he definido una hoja de ruta estratégica y técnica clara para las próximas fases:

1. **Estrategia de Datos y "Active Learning":** Para resolver los casos extremos (como porteros con equipaciones atípicas) sin etiquetar miles de fotos manualmente, implementaremos un flujo de aprendizaje activo. La idea es introducir pocas imágenes nuevas, dejar que el modelo actual las pre-etiquete y corregir solo los fallos. Esto acelerará exponencialmente la robustez del modelo ante nuevos equipos y pistas.
2. **Mejora de las fórmulas matemáticas:** Si bien el desarrollo actual cumple con los requisitos de un MVP, la fase de validación ha evidenciado que las fórmulas de proyección admiten una optimización significativa. Tal y como se ha discutido en el análisis de resultados, es prioritario refinar el algoritmo de estimación del Campo de Visión (FOV), ampliando el rango de búsqueda angular para ajustar con mayor fidelidad la lente específica de cada dispositivo móvil. Asimismo, la mejora más crítica será la integración del círculo central (*midfield*) como un nuevo "anclaje de profundidad"; esto permitirá recalibrar la coordenada Y de los jugadores más alejados mediante triangulación, corrigiendo la distorsión de perspectiva actual. Por último, se debe eliminar la restricción de máscara actual para permitir la proyección continua de eventos que ocurren fuera de los límites estrictos del terreno de juego (*out-of-bounds*) o en el interior de las porterías,

situaciones que actualmente el sistema ignora y que son vitales para el análisis de goles y jugadas a balón parado.

3. **Lógica de Tracking de Balón Contextual:** El balón sigue siendo el elemento más difícil de rastrear por su tamaño y velocidad. Implementaremos una lógica de búsqueda contextual que, cuando pierda el balón, lo busque priorizando las zonas cercanas al último jugador que lo tocó o calculando la física de su trayectoria probable, reduciendo así los falsos negativos.
4. **Consolidación del "Frame 0" y Partido Completo:** El objetivo inmediato es finalizar la lógica de inicialización para que el sistema sea capaz de mantener la identificación constante de los 10 jugadores durante todo el partido, gestionando las entradas y salidas de campo de forma automática.
5. **Validación en Entornos Hostiles:** Realizar una batería de pruebas de estrés ("Stress Testing") en pabellones con condiciones de iluminación deficientes, suelos de diferentes colores (parquet, goma azul) y grabaciones desde ángulos muy escorados, para garantizar que el producto funciona en la realidad del deporte municipal y no solo en vídeos ideales.
6. **Analítica Avanzada (Conexión con ADE):** Una vez que el tracking sea perfecto, el siguiente paso es explotar el dato. Se desarrollará el módulo de *Business Intelligence* imaginado en el prototipo del TFG de ADE, generando mapas de calor, estadísticas de posesión, kilómetros recorridos y redes de pases, aportando el valor táctico real que los entrenadores demandan.
7. **Viabilidad Comercial y Despliegue:** Finalmente, se explorarán herramientas de despliegue en la nube más eficientes (como ONNX Runtime o servicios *Serverless*) para exportar el modelo fuera de los cuadernos de investigación y convertirlo en una aplicación web o móvil viable económicamente.

11.Anexo

Anexo A: Imágenes Complementarias del Proceso.

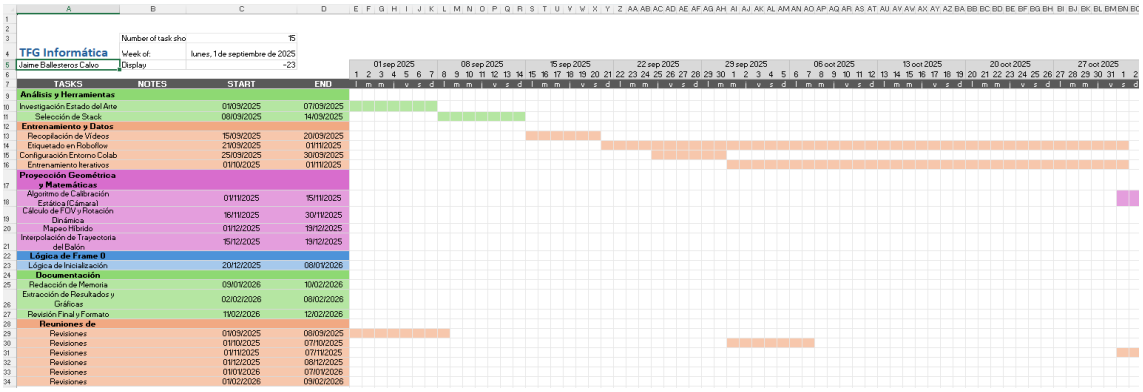

Ilustración 29. Gantt Chart Primeras Semanas

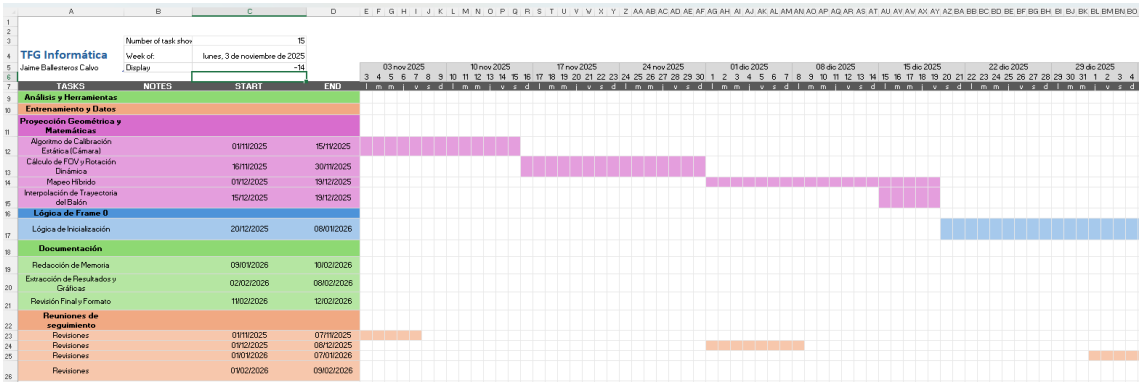

Ilustración 30. Gantt Chart Segundas Semanas

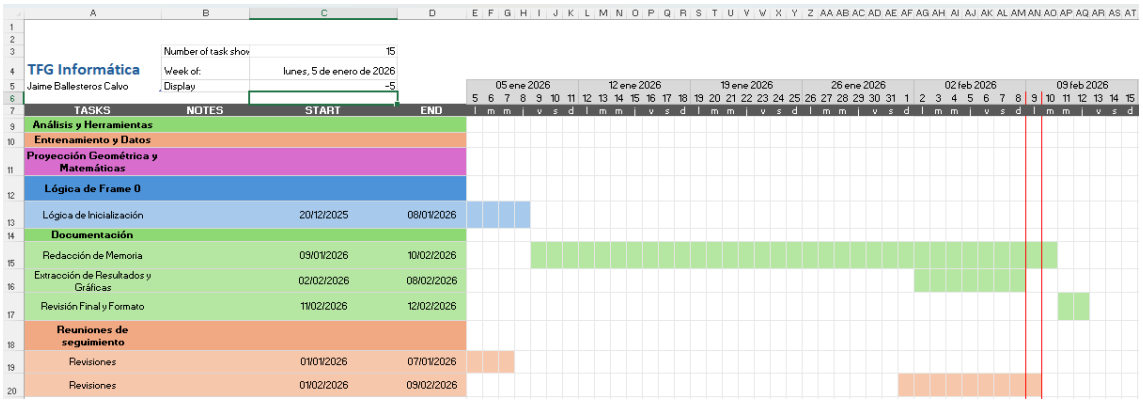

Ilustración 31. Gantt Chart Terceras Semanas

Anexo B: Repositorios de Código y Datasets

B.1 Github

URL:
https://github.com/JaimeBallesterosCalvo/TFG_Informatica_JaimeBallesterosCalvo

Descripción: En este repositorio se han guardado todos los archivos relevantes a este trabajo. Es por ello que hay 6 carpetas que se llaman:

- *Planificación***:** en donde está guardado el Gantt Chart
- *Resultados_Entrenamiento:* donde se guardan los csv con la información sobre los entrenamientos realizados en Google Colab
- *Resultados_Finales*: donde se guarda el video final y el frame 0.
- Codigo: donde se han adjuntado los 4 códigos que se han creado para este trabajo.
- *Diagramas*: en donde se han guardado el Diagrama de Casos, el Diagrama UML y el Diagrama del Sistema
- *Videos*: los dos vídeos utilizados para la creación de este MVP

B2. Roboflow

URL (las 5.000 imágenes de datasets):
https://universe.roboflow.com/tfgjaimeballesteroscalvo/tfg-jaime-ballesteros-calvo-rvc8d

URL (dataset utilizado para crear el modelo con 525 datasets):
https://universe.roboflow.com/tfgjaimeballesteroscalvo/tfg-jbc-final-data-3-xm1uc

12. Bibliografía

Agencia Española de Protección de Datos (AEPD). (2018). *Guía para centros educativos: Tratamiento de datos*. Recuperado de https://www.aepd.es/guias/guia-centros-educativos.pdf

Ballesteros Calvo, J. (2026). *Investigación de Mercado: Futsal Vision* [Trabajo de Fin de Grado no publicado]. Universidad Carlos III de Madrid.

Barça Innovation Hub. (s.f.). *Portfolio & Investment: Startups and Innovation*. Recuperado de https://barcainnovationhub.com/

Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLO* (Version 8.0.0) [Software de ordenador]. Recuperado de https://github.com/ultralytics/ultralytics

Kirillov, A., Mintun, E., Ravi, N., Mao, H., Rolland, C., Gustafson, L., ... & Girshick, R. (2023). Segment anything. *arXiv preprint arXiv:2304.02643*.

Metrica Sports. (s.f.). *Automated Tracking Data & Analytics*. Recuperado de https://metrica-sports.com/

Naciones Unidas. (2015). *Resolución 70/1. Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible*. Recuperado de https://www.un.org/sustainabledevelopment/es/

OLIVER Sports. (s.f.). *GPS Tracker for Football Performance*. Recuperado de https://oliversports.ai/

Real Federación de Fútbol de Madrid (RFFM). (s.f.). *Listado de Clubes*. Recuperado de https://www.rffm.es/competicion/clubes

RetailNext. (s.f.). *Traffic Counting & In-Store Analytics Technology*. Recuperado de https://retailnext.net/

Roboflow. (s.f.). *Licensing and Pricing Models*. Recuperado de https://roboflow.com/pricing

Tobii. (s.f.). *Eye tracking for research: How it works*. Recuperado de https://www.tobii.com/

Ultralytics. (s.f.). *Licensing: AGPL-3.0 & Enterprise*. Recuperado de https://www.ultralytics.com/license

Veo Technologies. (s.f.). *The ultimate camera for team sports*. Recuperado de https://www.veo.co/

13. Declaración de IA

He usado IAG en mi TFG

Marca lo que corresponda:

~~SI~~	NO

Parte 1: declaración sobre comportamiento legal, ético y responsable

Ten presente que el uso de IAG conlleva unos riesgos y puede generar una serie de consecuencias académicas graves: la evaluación de tu TFG puede verse comprometida si el uso de la IAG comporta la utilización de datos de carácter confidencial, materiales protegidos por derechos de autoría, o datos de carácter personal, y se hace sin cumplir las condiciones exigidas en cada caso (autorización de los interesados, autorización de los titulares, seguimiento de las instrucciones de la Universidad).

Pregunta
1. En mi interacción con herramientas de IAG he facilitado **datos de carácter confidencial** contando siempre con la debida autorización de los interesados. La confidencialidad abarca toda información que una persona u organización desea proteger por razones legales, comerciales, de privacidad o estratégicas (como patentes o secretos comerciales).

SÍ, he usado estos datos con la autorización de los interesados	~~NO, no he usado datos de carácter confidencial~~
2. En mi interacción con herramientas de IAG he facilitado **materiales protegidos por derechos de autoría** contando siempre con la autorización de los respectivos titulares.	
SÍ, he usado estos materiales con autorización de los titulares de derechos de autor; o bien sin ella porque se ajustan a una de las excepciones o límites que permite la ley: ● obra en dominio público ● obra licenciada (licencias Creative Commons) ● uso de fragmentos con fines de investigación (derecho de cita)	~~NO, no he usado materiales protegidos por derechos de autoría~~
3. En mi interacción con herramientas de IAG he facilitado **datos de carácter personal** con la debida autorización de los interesados.	
SÍ, he usado estos datos con autorización de los interesados y conforme a las instrucciones contenidas en la guía aprobada por la Universidad	~~NO, no he usado datos de carácter personal~~
4. Mi utilización de la herramienta de IAG ha **respetado sus términos de uso**, así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado, es decir, que produzca una impresión o conocimiento contrario a la realidad de los resultados obtenidos, que suplante mi propio trabajo o que pueda resultar en un perjuicio para las personas.	
~~SI~~	NO

Parte 2: declaración de uso técnico

Declaro haber hecho uso del sistema de IA Gemini para:

Documentación y redacción:

- *Soporte a la reflexión en relación con el desarrollo del trabajo: proceso iterativo de análisis de alternativas y enfoques utilizando la IAG*

N/A

- *Revisión o reescritura de párrafos redactados previamente*

Su uso se ha centrado principalmente en la revisión de los apartados con mayor carga técnica. En ocasiones, la redacción inicial de conceptos complejos —como la arquitectura de la red neuronal o los algoritmos de proyección geométrica— resultaba demasiado densa. He utilizado la herramienta para reformular estas explicaciones, buscando una mayor claridad expositiva y didáctica, pero supervisando siempre que se mantuviera el rigor y la precisión técnica de la definición original.

- *Búsqueda de información o respuesta a preguntas concretas*

Para la confección del Marco Teórico, se ha empleado el buscador de Google potenciado por inteligencia artificial como herramienta de exploración preliminar. Esta tecnología ha permitido agilizar la localización de bibliografía relevante, identificar formulaciones matemáticas específicas y descubrir enfoques arquitectónicos implementados en proyectos similares. No obstante, la IA ha actuado estrictamente como un mecanismo de filtrado y descubrimiento; la extracción de datos y la validación de la información se han realizado invariablemente consultando las fuentes primarias originales, garantizando así la veracidad de los conceptos y fórmulas adoptados.

Desarrollar contenido específico

Se ha hecho uso de IAG como herramienta de soporte para el desarrollo del contenido específico del TFG, incluyendo:

- *Asistencia en el desarrollo de líneas de código (programación)*

Se ha utilizado como recurso de apoyo técnico para resolver bloqueos puntuales en la implementación y sugerir mejoras en la eficiencia del código. La IA ha servido para contrastar alternativas de implementación y solucionar problemas de configuración de entorno, actuando siempre bajo supervisión y validación manual de cada línea integrada en el proyecto final.

- *Generación de esquemas, imágenes, audios o vídeos*

N/A

- *Procesos de optimización*

La IA ha actuado como una herramienta de revisión de código (Code Review), ayudando a optimizar la estructura y legibilidad de los scripts. Se utilizó para estandarizar el estilo de programación, eliminar redundancias y simplificar funciones complejas, asegurando que el proyecto siguiera principios de código limpio y mantenible.

- *Tratamiento de datos: recogida, análisis, cruce de datos…*

N/A

- *Inspiración de ideas en el proceso creativo*

En la fase de modelado matemático, la herramienta sirvió como un motor de búsqueda de enfoques alternativos para la proyección geométrica. Se utilizó para explorar distintas formulaciones teóricas (desde homografías simples hasta correcciones de distorsión no lineal), permitiendo contrastar rápidamente la viabilidad de cada método antes de su implementación.

Parte 3: reflexión sobre utilidad

En mi experiencia, la utilidad de estas herramientas ha sido transversal a todo el proyecto. En la fase de documentación, su principal fortaleza residió en la capacidad de desbloqueo a la hora de redactar, permitiéndome agilizar la escritura y corregir el estilo cuando tenía clara la idea pero no la forma exacta de expresarla; no obstante, he tenido que revisar minuciosamente cada texto para evitar el lenguaje artificial o repetitivo que a veces generan.

De igual forma, en el ámbito de la programación considero que es una herramienta indispensable y que, en la actualidad, no utilizarla supondría un error estratégico dada la ventaja competitiva que ofrece en velocidad. A pesar de ello, su uso no exime de la responsabilidad técnica: siempre es necesario comprobar el código generado ya que es propenso a fallos, y en muchas ocasiones, para lógicas muy específicas, resulta más rápido y sencillo implementar la solución manualmente que intentar corregir las iteraciones de la IA.